



University of
Sheffield

Evaluating Carbon-Aware Job Scheduling in HPC Using Trace-Driven Slurm Simulation

A Stanage Case Study

Yongjiang Liu

Supervisor: Tahsinur R Khan

Co-supervisor: Dr Fred Sonnenwald

MSc dissertation
School of Computer Science
University of Sheffield

September 2026

Declaration

All sentences or passages quoted in this report from other people's work have been specifically acknowledged by clear cross-referencing to author, work and page(s). Any illustrations that are not the work of the author of this report have been used with the explicit permission of the originator and are specifically acknowledged. I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this project and the degree examination as a whole.

Name:

Signature:

Date:

Abstract

Carbon-aware scheduling can delay computing work until electricity has a lower carbon intensity. In a shared HPC cluster, however, changing a release time also changes queue competition. An apparent carbon benefit may come with additional waiting, concentrated user costs or dependence on an uncertain power model.

This dissertation evaluates carbon-aware job admission using the University of Sheffield’s Stanage service as a case study. An anonymised archive of 4.11 million job records supports workload analysis and trace-driven Slurm simulation. Seven representative admission rules are compared on five matched UTC dates, with three baseline and two Adaptive Balanced replays per date, giving 55 final completed runs. The analysis combines regional carbon intensity with a stated cage-power scenario, and reports carbon change alongside paired waiting, user burden and three resource-occupancy representations.

The results vary by date and model. B1 reduces the primary load-dependent carbon estimate on four dates, but a 2.6427% worst-day increase leaves its equal-day mean reduction negative. History-only has a small positive mean reduction of 0.1021%, accompanied by 4.97 minutes of mean additional total waiting per evaluation job. Its burden is uneven: on one date, a single user’s 35 jobs account for 86.77% of direct policy delay. Adaptive’s runtime-related budget constrains intentional delay, but does not guarantee short subsequent queueing. No configuration satisfies every frozen stability condition. No-action replays and timing audits show why small numerical differences cannot automatically be attributed to a scheduling intervention.

A separate bounded extension implements feasibility-first selection, cumulative user budgets and feedback from actual simulator events. Its functional check demonstrates that cancelling remaining delay can still leave a job behind later arrivals; its 64-job trace sample takes no hold action. The contribution is an implemented, auditable evaluation of carbon and service trade-offs, together with tested engineering improvements. The findings concern modelled operational carbon and do not claim measured Stanage electricity savings.

Acknowledgements

I would like to thank my supervisor, Tahsinur R Khan, for his guidance and feedback throughout this project. Our meetings helped me turn a broad interest in HPC carbon monitoring into a more focused study of scheduling, waiting time and the limits of carbon estimates. His questions about the data and equations encouraged me to explain what the results actually support.

I also thank my co-supervisor, Dr Fred Sonnenwald, and the University of Sheffield Research IT Services team. Fred provided the Stanage job archive and helped clarify the cluster configuration, the available energy information and the practical context of the work. This support made the trace-driven case study possible.

Contents

Acknowledgements	iii
1 Introduction	1
1.1 Background	1
1.2 Problem and motivation	1
1.3 Aim and objectives	2
1.4 Research questions	2
1.5 Contributions and scope	3
1.6 Dissertation structure	3
2 Background and Literature Review	4
2.1 Energy, carbon and the measurement boundary	4
2.2 Temporal flexibility and carbon information	6
2.3 Carbon-aware scheduling and Slurm admission	7
2.4 Waiting budgets and the distribution of service costs	8
2.5 Trace-driven simulation and experimental credibility	9
2.6 Planning and reporting a computational study	10
2.7 Synthesis and research position	10
3 Requirements, Data Analysis and Methodology	12
3.1 Research design and requirements	12
3.2 Accounting archive and privacy	13
3.3 Exploratory data analysis	15
3.4 Daily cohorts and carry-in reconstruction	17
3.5 Simulated cluster and flexibility	19
3.6 Policies and information access	19
3.7 Operational carbon model	20
3.8 Service metrics and user burden	24
3.9 Repetition and multi-day interpretation	26
3.10 Validity and completion requirements	28

4	Design and Implementation	29
4.1	Pipeline architecture	29
4.2	Trace conversion and event generation	29
4.3	Slurm integration	31
4.4	Admission rules	31
4.4.1	Normal admission and runtime-aware shifting	33
4.4.2	High-carbon admission gate	33
4.4.3	Adaptive Balanced	33
4.4.4	Direct day/night hybrid	36
4.4.5	Oracle load information	36
4.4.6	History-only decisions	36
4.5	Accounting and audit outputs	37
4.6	Multi-day execution	37
4.7	Engineering extension after the frozen comparison	38
4.7.1	Candidate selection and budget accounting	38
4.7.2	Connection to actual simulator events	39
4.7.3	Bounded checks and their scope	41
5	Evaluation and Results	42
5.1	Evidence stages	42
5.2	Historical comparisons and corrections	42
5.2.1	Queue fidelity and the November comparison	42
5.2.2	Why the old December percentage is not retained	46
5.2.3	Historical predictor diagnostics	47
5.3	Completed five-date comparison	47
5.3.1	Direction, scale and the meaning of an average	50
5.3.2	Waiting and concentrated user burden	53
5.3.3	No-action cases and repeat variation	56
5.3.4	Stability under the frozen criteria	58
5.4	Explaining observed carbon increases	58
5.5	Completion, timing and retained failures	60
5.6	Bounded feedback-extension results	60
5.6.1	Code behaviour and the functional fixture	60
5.6.2	The paired 64-job trace sample	62
6	Discussion	63
6.1	Answers to the research questions	63
6.1.1	RQ1: Changes in modelled carbon	63
6.1.2	RQ2: Waiting and user burden	64
6.1.3	RQ3: Information and delay safeguards	64
6.1.4	RQ4: Consistency across dates and assumptions	65
6.2	What the carbon model supports	66

6.3	Replay validity and generalisation	67
6.4	What the feedback extension adds	67
6.5	Practical implications and future work	68
7	Conclusion	69
	Appendices	75
A	Reproducibility and Evidence Guide	76
B	Complete five-date results	78

List of Figures

3.1	Annual data processing and daily cohort construction	14
3.2	Monthly workload and completion proportion	15
3.3	Annual runtime and waiting percentiles	16
3.4	Submission patterns by UTC hour and weekday	16
3.5	Paired replay and common accounting design	18
3.6	Prepared five-date workload cohorts	18
3.7	Historical regional carbon-intensity inputs	22
3.8	Occupancy representations and carbon-accounting boundary	23
3.9	Job timeline and waiting-time components	25
3.10	Five-date replay allocation	27
4.1	Implemented simulation and analysis pipeline	30
4.2	B1, B2 and Adaptive release mechanisms	32
4.3	Adaptive Balanced decision flow	35
4.4	Event-driven admission feedback	40
5.1	Source-trace and replay queue fidelity	43
5.2	November carbon and waiting trade-offs	45
5.3	November distribution of policy-delay burden	46
5.4	Later-month predictor coverage	47
5.5	Complete daily primary carbon results	49
5.6	Per-policy daily carbon and paired wait	51
5.7	Complete per-date carbon, mean wait and tail wait	52
5.8	Daily user burden	54
5.9	Direct holds, actual added wait and indirect waiting	55
5.10	Replay variability and model direction sensitivity	57
5.11	Within-window carbon increase decomposition	59
5.12	Queue-position cost in the functional fixture	61

List of Tables

2.1	Related work and its relevance to admission control	11
3.1	Archive processing and data quality	13
3.2	Frozen policy information and key parameters	20
3.3	Symbols and units in the carbon calculation.	24
3.4	Five-date cohort inventory	26
4.1	Implemented improvements and the evidence required to assess them.	38
5.1	Experimental stages and their role in the dissertation.	43
5.2	November policy comparison	44
5.3	Five-date primary policy summary	48
5.4	V2 paired 64-job engineering check. Waiting and runtime errors are in seconds.	62
B.1	All 35 daily carbon results (2025). $K_C = C_{b,C} - C_{p,C}$ is primary cluster saving (kg CO ₂); $g_C = 1000K_C/N$ (g CO ₂ per evaluation job). R_C, R_R, R_A are load-related saving percentages for capacity-weighted, node-request upper and distinct allocated-node proxies; $R_{C,w}$ is whole-cluster primary saving percentage. Negative means increase. † marks zero-action replay variability. Policy abbreviations follow the results chapter.	79
B.2	All 35 daily wait and burden results (2025). n_D counts directly held jobs; $\bar{D}$ is mean hold. $\overline{\Delta W}$ and $P95(\Delta W)$ summarize paired added total wait; ΔW_0 is the mean among jobs not directly held. U_D, U_+ are the largest user's shares of total hold and positive added wait. Times are minutes per job in the stated cohort. † marks zero-action replay variability.	81

Abbreviations

HPC	High-performance computing
CI	Carbon intensity
EDA	Exploratory data analysis
NESO	National Energy System Operator
PUE	Power usage effectiveness
UTC	Coordinated Universal Time
CPU / GPU	Central / graphics processing unit

Chapter 1

Introduction

1.1 Background

High-performance computing supports research whose computational demands exceed the capacity of an individual workstation. A shared cluster allows many researchers to use CPU and GPU resources, but those resources consume electricity while running jobs and while waiting for work. Estimating the environmental consequence of computation therefore requires a definition of what equipment is included, how energy is obtained and which electricity carbon intensity is applied. Computational footprint methods make these choices explicit because the same runtime or resource request does not identify a unique carbon footprint [1, 11].

Electricity carbon intensity also changes over time. For a job that can tolerate a later start, moving computation to a lower-intensity interval may reduce its estimated operational emissions. Temporal shifting has consequently been studied in cloud and cluster settings [25, 29]. The opportunity is constrained: a long job may continue into a higher-intensity period, a delayed job can compete with later arrivals, and resource allocation does not necessarily correspond to measured electrical load. A useful evaluation must consider these effects together with the waiting imposed on users.

This project studies those questions using Stanage, the University of Sheffield HPC service, as a case study. Stanage uses Slurm to manage batch work across heterogeneous resources [26]. Slurm retains responsibility for priority, resource feasibility and job starts in the experiments. A separate admission layer changes when selected jobs become visible to it. This division makes it possible to investigate a carbon-aware intervention without claiming to replace the cluster scheduler.

1.2 Problem and motivation

The available Stanage archive contains job timestamps and resource allocations, but it does not provide usable per-job energy measurements. Approximate power figures supplied during project supervision describe equipment inside the Stanage cage. They support a stated power scenario; they do not establish an electrical model for each individual job. The methodological

task is therefore to build controlled comparisons and expose their assumptions rather than attach an apparently exact footprint to an unmeasured workload.

The scheduling task is equally important. A policy could lower an emissions estimate by imposing substantial delays, concentrate most of those delays on a small number of users, or appear favourable only under one utilisation proxy. A carbon percentage alone cannot distinguish these cases. Conversely, a policy that deliberately abstains from delaying uncertain jobs may provide a useful service safeguard even when it achieves no reduction on a particular date. The project evaluates the trade-off instead of defining success as the largest isolated saving.

Earlier project work established a functioning simulator and developed candidate admission rules. The final comparison evaluates seven representative policies on the same workloads across five dates, with repeated baselines. A separate, bounded engineering extension then tests candidate selection, user budgets and feedback from the actual simulated queue. This dissertation connects those implementations to an evaluation that keeps information access, waiting cost and model uncertainty visible throughout the comparison.

1.3 Aim and objectives

The aim is to evaluate the carbon and service consequences of carbon-aware job admission through trace-driven Slurm simulation under limited energy telemetry.

The objectives are to audit the supplied accounting archive; reconstruct evaluation workloads and their initial queue pressure; implement representative admission policies; estimate operational carbon using a transparent and consistent accounting boundary; compare service outcomes and user delay; and examine the stability of those comparisons across dates, simulator repeats and resource-occupancy models. Each objective is tied to an observable output: cleaned aggregates, workload manifests, policy decisions, completed replay records or comparative results.

1.4 Research questions

The following questions organise the dissertation. They consolidate the questions developed during the project without changing the experimental scope.

1. How do representative carbon-aware admission policies change modelled operational carbon for a matched Slurm workload?
2. What waiting and user-burden costs accompany these changes, and how does the adaptive waiting budget affect that trade-off?
3. How do policies with different runtime/load information and delay safeguards compare, including variants that use estimates from earlier months?

4. How consistent are the carbon and service outcomes across dates, replay variation and occupancy-model assumptions?

The questions are related but require separate answers. A carbon reduction need not imply a service improvement. A decision made with historical runtime estimates need not have used a genuinely available forecast of grid intensity. A technically complete replay need not establish a robust policy benefit.

1.5 Contributions and scope

The implementation provides a path from an anonymised accounting archive to trace-derived events, repeatable experimental inputs and paired carbon/service analysis. It includes reconstruction of running, queued and held carry-in work; admission rules with different delay safeguards and information requirements; and checks that separate workload validity from timing fidelity and policy outcomes. The main empirical contribution is a completed comparison of seven rules on five dates, comprising 55 final replays. It identifies date-dependent carbon changes, concentrated user delay and differences between occupancy models. A later feedback prototype makes candidate feasibility, shared user budgets and cancellation of remaining admission delay explicit in the code. Its small tests establish mechanism behaviour rather than another five-date performance result.

The study is retrospective. The simulator executes the supplied or capped runtime so that jobs finish, while policy-specific rules determine which runtime information may be used to decide a delay. Historical regional carbon series are shared across policies. The absence of archived decision-time forecast vintages means that even the history-only scheduling variants do not establish a complete online carbon-forecast evaluation.

The primary model covers operational electricity within the stated cage boundary. Embodied emissions, hardware manufacture and disposal are excluded; lifecycle accounting requires additional evidence [8]. No new production Slurm configuration, hardware power cap, checkpointing or preemption is deployed. The simulated flexible subset is an experimental assumption rather than a record of individual users agreeing to delayed service.

1.6 Dissertation structure

Chapter 2 reviews carbon accounting, temporal shifting and shared-cluster evaluation. Chapter 3 describes the data and experimental design. Chapter 4 explains workload reconstruction, admission policies and the later feedback extension. Chapter 5 presents the historical development evidence, completed five-date comparison and bounded extension tests. Chapter 6 answers the research questions and examines the limits of the evidence. Chapter 7 summarises the contribution and possible future work. The appendices provide the evidence guide and complete daily numerical tables.

Chapter 2

Background and Literature Review

2.1 Energy, carbon and the measurement boundary

Energy efficiency and carbon efficiency describe related objectives. A system can consume less electricity by using its hardware more efficiently, or it can change the timing of consumption to coincide with a lower-carbon electricity supply. Surveys of energy-aware HPC distinguish scheduling, application optimisation, frequency scaling and power capping, while also documenting the difficulty of modelling heterogeneous processors and accelerators accurately [6, 18]. This distinction determines the scope of the present study: an admission policy changes when a fixed workload becomes eligible for execution. It does not itself implement a device power limit or demonstrate that the workload needs less energy.

Zeus provides a concrete example of the separate energy-control problem. It changes batch size and GPU power limits to navigate an energy-training-time trade-off for recurring DNN jobs [32]. Lower instantaneous power need not minimise total energy if execution becomes longer, so its control objective considers both. The Stanage admission rules retain resources and runtimes. They can change the time at which a fixed workload consumes energy in the scenario model, but they do not establish the application-level energy reduction studied by Zeus.

Carbon-responsive device control occupies another point in this design space. Chase adapts GPU power limits during execution using forecast carbon intensity [31]. This changes training progress and energy use inside an already running job, whereas admission shifting changes when that job becomes eligible to run. Both can respond to electricity conditions, but their service costs are expressed through different mechanisms. This distinction prevents a comparison between a Slurm waiting-time change and a DNN training-time penalty from being treated as if the two were measured on the same workload.

Operational carbon accounting combines energy consumption with an electricity-emissions factor. Green Algorithms estimates computational energy from information such as runtime, processor characteristics and allocated memory, including assumptions about utilisation and facility overhead [11]. Its accessibility is valuable when direct measurements are unavailable, but the result depends on the adequacy of those coefficients. Carbontracker instead samples

supported hardware interfaces during execution and combines energy estimates with carbon information [1]. Even this approach requires a declared device boundary: monitoring a GPU does not automatically measure the host CPU, storage, networking or cooling.

Trace-based tools provide a third useful perspective. Ichnos aligns scientific-workflow traces with resource power models and time-varying carbon intensity, demonstrating how post-hoc estimation can use existing execution records [28]. Its accuracy is assessed for its own resources and workloads; that validation cannot be transferred to Stanage without comparable measurements. Likewise, the F-DATA release illustrates the richer job, power and energy information available on Fugaku, rather than supplying missing Stanage telemetry [2]. For this dissertation, allocated resources are observable scheduling quantities and proxies for activity, not measurements of electrical utilisation.

These approaches also answer different validation questions. A coefficient model can be checked for consistent units and sensitivity to its inputs, whereas a meter-based estimator can additionally be compared with an independent electrical measurement. A trace estimator needs both a credible power mapping and sufficiently precise execution intervals. The survey by O’Brien and colleagues shows why hardware and application characteristics matter to predictive models [18]. Consequently, resource-count agreement between a generated workload and its replay establishes workload consistency; it cannot, by itself, validate the estimated kilowatt-hours.

The practical comparison is therefore between evidence requirements, not simply between convenient tools. Green Algorithms can operate from a small set of workload descriptors, while Carbontracker requires access to supported device measurements during execution [1, 11]. Stanage accounting already provides descriptors and intervals but lacks usable per-job energy. This makes a declared scenario feasible for the present retrospective study. A future metered evaluation would have to align measurement scope, sampling interval and idle allocation with that scenario before comparing the two estimates.

The wider modelling literature reinforces the importance of this alignment. Kamiya and Coroamă’s technical review identifies system boundaries, source data and modelling assumptions as reasons that data-centre energy estimates diverge [10]. Comparing estimates without those details can confuse a change in the accounting boundary with a change in efficiency. For this dissertation, the relevant consequence is to state that the supplied power figures describe equipment inside the cage and to examine alternative allocation representations within that same boundary. Global data-centre estimates in the review do not calibrate Stanage’s coefficients.

The system boundary also determines what a reported reduction means. ACT and the HPC lifecycle analysis by Li and colleagues distinguish operational emissions from manufacturing-related embodied emissions [8, 13]. Rescheduling a job on existing hardware does not establish a reduction in its manufacturing footprint. The present evaluation therefore concentrates on an operational electricity scenario. Relative comparisons remain conditional on that model: common assumptions improve comparability, but do not ensure that model errors cancel when policies produce different resource placements.

The BLOOM footprint study illustrates the importance of stating which components a reported total contains. It separates dynamic consumption, idle infrastructure and embodied emissions [14]. These components need different evidence and allocation assumptions. A scheduling intervention can change the temporal exposure of running work without changing hardware manufacture or eliminating idle infrastructure. The present distinction between load-dependent and whole-cage carbon serves the same purpose of making the reported quantity explicit; neither BLOOM’s component ratios nor its training-utilisation assumptions are transferred to Stanage.

2.2 Temporal flexibility and carbon information

Temporal shifting uses the fact that electricity carbon intensity varies during a job’s feasible execution period. Wiesner and colleagues [29] examine how flexibility windows, interruption assumptions and forecast accuracy affect this opportunity. For a non-preemptive job, the relevant exposure extends across its runtime. A favourable intensity at release can be followed by several unfavourable hours, so selecting the lowest single timestamp is only an approximation to the interval objective. This motivates comparing a simple time-shifting baseline with a policy that evaluates the complete candidate runtime.

The opportunity is constrained by the workload. Sukprasert and colleagues [25] analyse ideal and practical shifting bounds across many regions and show that attainable benefits depend on job duration, deadlines and service constraints. Their findings motivate retaining simple baselines: a more elaborate decision rule should justify its complexity through an improved trade-off. They also caution against interpreting a large saving in one setting as a general property of carbon-aware computing. In a university cluster, carbon variation must coincide with both available capacity and jobs whose users can tolerate a delay.

A flexibility window is also distinct from an application deadline. A maximum admission delay constrains the policy’s own action; it does not constrain queueing after release, and it does not guarantee a completion time. The shifting analysis by Wiesner and colleagues explicitly considers the conditions under which work may be deferred or interrupted [29]. The present policies use non-preemptive execution and a bounded release change. This narrower control surface explains why responsiveness has to be measured after replay even when every admission decision satisfies its budget.

There are two useful comparison levels. An opportunity analysis asks whether some later execution interval has a lower carbon exposure for the same work. A scheduler evaluation asks whether the resulting set of admission decisions still reduces exposure once jobs compete for capacity. The limitations identified by Sukprasert and colleagues show why practical restrictions can reduce ideal shifting potential [25]. In this project, the first level is recorded in the decision audit and the second in the completed schedule. Keeping both permits a negative or small realised change to be investigated rather than attributed automatically to a calculation error.

NESO provides the regional signal used here. Its documentation describes electricity-generation CO₂ intensity, regional modelling and half-hour updates; the API identifies South Yorkshire as region 5 [16, 17]. A regional value is neither a site electricity meter nor a complete lifecycle greenhouse-gas inventory. An additional distinction concerns information timing. A historical series obtained after an event does not by itself establish which forecast was available when a scheduling decision would have been made. Consequently, retrospective carbon exposure, forecast-informed decisions and empirical forecast-error evaluation require separate descriptions. Synthetic perturbations can test sensitivity, but they cannot substitute for matched archived forecasts and later estimates.

Another distinction concerns the emissions question being asked. Dodge and colleagues distinguish marginal-emissions accounting, which addresses the change associated with an intervention, from attributional accounting based on average electricity intensity [7]. The NESO series used here is a regional generation-intensity signal. Multiplying modelled consumption by that signal attributes an operational footprint to the simulated schedule; it does not measure the additional generator that responds to a particular admission decision. The term “carbon reduction” in the results therefore refers to a paired reduction under the stated attributional model. It should not be read as a validated causal change in grid emissions.

2.3 Carbon-aware scheduling and Slurm admission

GREEN provides a closely related example of integrating carbon objectives with cluster scheduling [30]. It combines ML-job energy information with temporal flexibility, resource allocation and mechanisms intended to protect completion time. Its implementation includes active scheduling rounds, job scaling and checkpoint-based preemption. These capabilities explain both its relevance and its limits as a comparator. Stanage’s historical mixed workload does not provide equivalent job energy profiles or evidence that arbitrary jobs can resize or restart safely. The transferable ideas are to account for carbon over execution, consider resource pressure and protect waiting time; its reported savings are not a benchmark that the Stanage admission policy can be expected to reproduce.

PCAPS makes another constraint explicit: delaying one task may block dependent tasks and extend the completion of an entire data-processing job. It combines precedence information with a configurable carbon-versus-completion-time objective [12]. This supports transparent trade-off parameters, although the DAG information used by PCAPS is different from the eligibility and resource fields available here. The Stanage study preserves observed eligibility as the boundary after recorded dependencies or holds and does not claim to reconstruct the full application dependency graph.

A lighter integration model is illustrated by CATS, a Python tool for choosing lower-carbon task execution periods [3]. Its current documentation includes use with `sbatch` [4]. Slurm’s own `--begin` option defers allocation eligibility; it does not promise that a job starts at the specified time [19]. This distinction is central to the dissertation. An external admission layer chooses a release time, while Slurm determines the actual start and placement. The emissions

consequence must therefore be calculated from the realised simulated schedule, rather than assumed from the selected release alone.

Deeper changes to Slurm are also possible. Chadha and colleagues [5] extend it to schedule malleable applications that can change their resource allocation at runtime. That work demonstrates a different intervention with different application requirements. Keeping the historical Stanage jobs rigid allows the present experiment to isolate admission timing, while leaving resizing, preemption and hardware power control outside the implemented policy family.

A comparison of these schedulers should preserve their optimisation variables. GREEN can change the amount and timing of ML resources, PCAPS reasons about precedence-constrained tasks, and CATS provides a lightweight task-timing interface [3, 12, 30]. Their mechanisms are informative even though they do not form a common benchmark with the same jobs, resource model and service constraints. Reproducing a published reduction percentage would require reproducing those experimental conditions. The present study instead tests a limited set of transferable ideas through a shared admission interface, so a policy comparison has a common intervention boundary.

2.4 Waiting budgets and the distribution of service costs

Carbon reduction becomes a service-quality question when it is obtained through delay. Starburst, although designed for hybrid-cloud cost control, offers a useful waiting-budget framework for exposing this trade-off [15]. It assigns different waits to jobs with different resource demands and allows administrators to choose a cost-completion-time balance. The principle can inform a carbon-aware policy without importing its cloud-bursting assumptions. In particular, a bounded allowance can prevent a short job from receiving a delay that is disproportionately large relative to its execution time.

This motivates the Adaptive policy in the present project: retain carbon-sensitive admission while adding a runtime-related waiting budget, a minimum expected benefit and an explicit waiting penalty. These are design choices informed by the literature, rather than a reproduction of Starburst or GREEN. Their effectiveness must be tested through the common Slurm replay. A budget limits the delay intentionally added by the admission layer; queue interactions can still affect other jobs after the workload is reordered.

Aggregate waiting measures alone cannot describe the resulting distribution. A lower mean can coexist with an increased tail, and a favourable overall percentile can conceal a burden concentrated on one user. The evaluation therefore reports waiting percentiles, slowdown and user-level delay beside modelled carbon. The purpose is to describe a defensible balance among carbon, responsiveness and fairness. A setting that benefits one objective remains informative even when it is not preferred under another; it should not be reduced to a single unqualified success or failure label.

Waiting budgets do not make unlike objectives interchangeable. Starburst’s cost-versus-completion-time control motivates explicit service preferences, while PCAPS illustrates why

the completion effects of delaying a task depend on surrounding work [12, 15]. A weighted policy score is consequently a decision rule, not a complete evaluation metric. Carbon, total waiting and user burden must still be reported in their original units. This permits a reader to see whether a favourable score was obtained through a material service cost and whether different users experienced that cost unevenly.

User-level carbon attribution introduces a further, separate choice. Wassermann and colleagues distinguish operator-level reporting from user-centred accounting and make the allocation of idle-resource carbon explicit [27]. Their framework connects measurement and resource use to accounting responsibilities. By contrast, the present user metrics allocate deliberate waiting, not a carbon bill. A user who receives much of the policy delay is not necessarily responsible for a corresponding share of cage emissions. Maintaining that distinction allows burden concentration to be discussed without assigning an unmeasured individual environmental footprint.

2.5 Trace-driven simulation and experimental credibility

The Slurm simulator lineage offers a way to examine scheduling changes without intervening in production. The earlier simulator retains a real Slurm instance with modifications for historical-job replay and faster execution [23]. The later work by Simakov and colleagues [22] emphasises careful transformation of Slurm code and the variability of scheduling realisations. Separately, Jokanovic and colleagues [9] compare a Slurm simulator with a physical installation. Such studies support the evaluation approach while also showing why fidelity must be established for a particular simulator, configuration and workload.

The scheduling configuration can affect outcomes even when workload membership is fixed. Simakov and colleagues study how node sharing and controller configuration change scheduler responsiveness and waiting in Slurm replay [21]. This makes placement and sharing relevant to both service and occupancy analysis. A sum of requested nodes, for example, does not establish the number of distinct hosts active when jobs can share nodes. The current sensitivity calculations retain these different views, while the experiment keeps its chosen Slurm configuration fixed across policies.

Slurm backfill considers requested runtime and resources when deciding whether lower-priority work can proceed without delaying higher-priority reservations [20]. Admission changes can therefore alter interactions among jobs, even when each job’s resources and runtime remain fixed. The current Stanage documentation confirms a heterogeneous CPU/GPU environment and Slurm-based submission, but it describes the current service and cannot alone establish the complete 2025 hardware inventory or production configuration [26]. Site history, reservations and administrator actions remain separate evidence requirements.

For this project, experimental credibility rests on explicit controls: common job membership, fixed resource requests and runtimes, reconstructed carry-in state, complete execution and a common carbon-accounting horizon. These controls make policy comparisons interpretable. They do not establish that simulated waiting reproduces production waiting, nor

does a structurally valid run establish a carbon benefit. Repeated baseline runs and checks across dates are particularly important when the estimated policy difference is small.

Simulator validity has several layers. Input checks establish that the intended job population and resource requests were encoded correctly. Execution checks establish that those jobs were submitted, scheduled and completed. Comparison with a physical system addresses a further question: whether the simulator reproduces the scheduling behaviour of that system under comparable conditions. Jekanovic and colleagues conduct this latter kind of validation for their simulator setting [9]. Passing the first two layers here does not inherit the third from their paper.

The simulator version and configuration are also part of the experimental method. The earlier Slurm simulator study explores implementation and parameter effects, while the later simulator work identifies sources of variation in scheduling realisations [22, 23]. This motivates saving configuration and workload hashes alongside results and repeating the baseline used for each date. A small carbon difference whose sign changes across repeated schedules requires a more cautious interpretation than a difference that remains outside the observed repeat range. The completed comparison in Chapter 5 applies this distinction to the saved outcomes.

2.6 Planning and reporting a computational study

The evaluation process itself also needs a clear evidence trail. Research-computing guidance from Souter and colleagues recommends planning analyses to reduce avoidable reruns while retaining useful replication and distinguishing planned work from subsequent exploration [24]. This is relevant to a project in which pilot windows were used to develop policies before a later multi-date campaign was frozen. A recorded plan identifies the intended comparison; it does not make earlier development data an untouched test set. Chapter 5 separates historical diagnostics, the completed main comparison and the later bounded engineering extension.

Transparent reporting connects a footprint estimate to its calculation method and execution context [24]. For this study, that requires identifying the source workload, policy parameters, power scenario, carbon series and accounting horizon. A second accounting problem would be the footprint of running the research experiments themselves. The simulated Stanage footprint is a counterfactual output and cannot substitute for the actual electricity consumed by the machine hosting the simulator. No estimate of that separate experimental footprint is claimed without the corresponding execution and energy records.

2.7 Synthesis and research position

The literature motivates a joint evaluation of carbon-aware admission: transparent energy boundaries, full-runtime exposure, capacity awareness, waiting safeguards and controlled replay. These principles still need to be tested together on a mixed university workload.

Table 2.1 brings these distinctions together. The useful comparison is the intervention each system can make and the information it requires. This makes the scope of an admission-only experiment visible before its results are interpreted.

Table 2.1: Selected related work compared by intervention and information needs. The final column states the design relevance to this project; it is not a comparison of measured savings across systems.

Work	Intervention and information	Relevance to the Stanage study
Let’s Wait Awhile [29]	Execution timing within a flexibility window; job duration and carbon information.	Evaluate exposure over runtime and state the delay allowance and forecast assumptions.
GREEN [30]	ML-cluster scheduling with energy profiles, scaling and checkpoint-based preemption.	Consider carbon and capacity together; application resizing and checkpointing are outside the implemented admission layer.
PCAPS [12]	Carbon-aware scheduling with task-precedence information and a completion-time trade-off.	Make service costs explicit; observed eligibility is available here, but a complete task DAG is not.
Starburst [15]	Job-sensitive waiting budgets for hybrid-cloud cost and completion-time control.	Motivate bounded waiting; cloud migration and cost optimisation are not part of this experiment.
CATS [3, 4]	Lightweight carbon-aware task timing, including documented sbatch integration.	Separate admission eligibility from the actual allocation and start chosen by Slurm.
This study	Fixed-job release-time policies with declared carbon/load information, carry-in and matched Slurm replay.	Compare modelled carbon, waiting and user burden. Multi-day outcomes remain to be completed.

This dissertation compares normal admission, B1 shifting, B2 gating and Adaptive admission with explicit waiting protection. Oracle and history-based variants add comparisons under different information and safeguards. The multi-day evaluation will determine how consistently carbon, waiting and user burden change together; its findings remain open.

Chapter 3

Requirements, Data Analysis and Methodology

3.1 Research design and requirements

The study uses controlled trace-driven simulation. Within an evaluation date, the baseline and all policy scenarios contain identical jobs, requested resources, simulation runtimes, pseudonymous users, flexibility assignments and carry-in work. Only the release decisions change. This design isolates a policy intervention within the model; it does not claim that the model recreates every aspect of production Stanage.

Four requirements guide the pipeline. First, every result must be traceable to its workload and policy parameters. Second, all expected jobs must complete, so apparent savings cannot arise from missing or timed-out work. Third, carbon accounting must include the delayed execution tail and use a common horizon within each comparison. Fourth, carbon, service impact and model sensitivity must remain separately inspectable. These requirements address all four research questions in Chapter 1.

The validation requirements follow the distinction between a functioning replay and a demonstrated reproduction of a physical installation in simulator research [9]. Here, the former is checked directly. The latter remains limited by the historical configuration and telemetry available for Stanage.

The primary simulator is the Slurm simulator associated with Simakov et al. [22]. It allows the project to retain Slurm scheduling logic while replacing wall-clock time and job execution with simulated events. FastSim was investigated during initial development but was not retained in the final evaluation pipeline. This avoids combining results from engines with different input assumptions before either could be validated against the available case-study data.

3.2 Accounting archive and privacy

The supplied 2025 archive contains 4,114,698 raw rows. Deduplication and the retained-year processing produce 4,098,801 jobs associated with 929 anonymised users. These are archive-level counts, not the number of jobs in an individual policy experiment. The preparation stabilises duplicated column names, parses the pipe-delimited records and interprets timestamps in UTC. Duplicate records for numeric job identifiers across monthly files are resolved by retaining the latest record.

The relevant fields include submission, eligibility, start and completion timestamps; state; partition; requested and allocated CPU, node, memory and GPU resources; and runtime or time-limit fields. Job names, commands, paths and research content are not needed for the comparative analysis. Window-local pseudonyms permit user-burden analysis without exporting raw user identities. Figures and tables in this dissertation use aggregate summaries; the raw archive and private local configuration are not part of the writing package.

Table 3.1: Accounting archive processing and selected quality exceptions. Exception rows overlap; only the first six rows describe the sequential cohort accounting.

Processing step or exception	Count
Raw accounting rows	4,114,698
Duplicate rows removed	14,576
Invalid JobIDRaw excluded	1
Globally unique valid jobs	4,100,121
Outside 2025 or missing Submit	1,320
Retained 2025 submission cohort	4,098,801
Missing Start in retained cohort	92,536
Negative eligible wait	7
Invalid requested GPU counts	2

Submission and eligibility have different meanings. A job can be submitted but unavailable to the scheduler while waiting for a dependency or an administrative hold. The baseline admission event is therefore anchored to **Eligible**, and dependency/hold time is kept as a separate component of user waiting. Treating all submission-to-start time as scheduler delay would assign the policy responsibility for time outside its control.

Figure 3.1 separates the annual descriptive population from the daily replay population. The latter is reconstructed from eligibility and boundary state rather than obtained by simply taking one day of annual submissions.

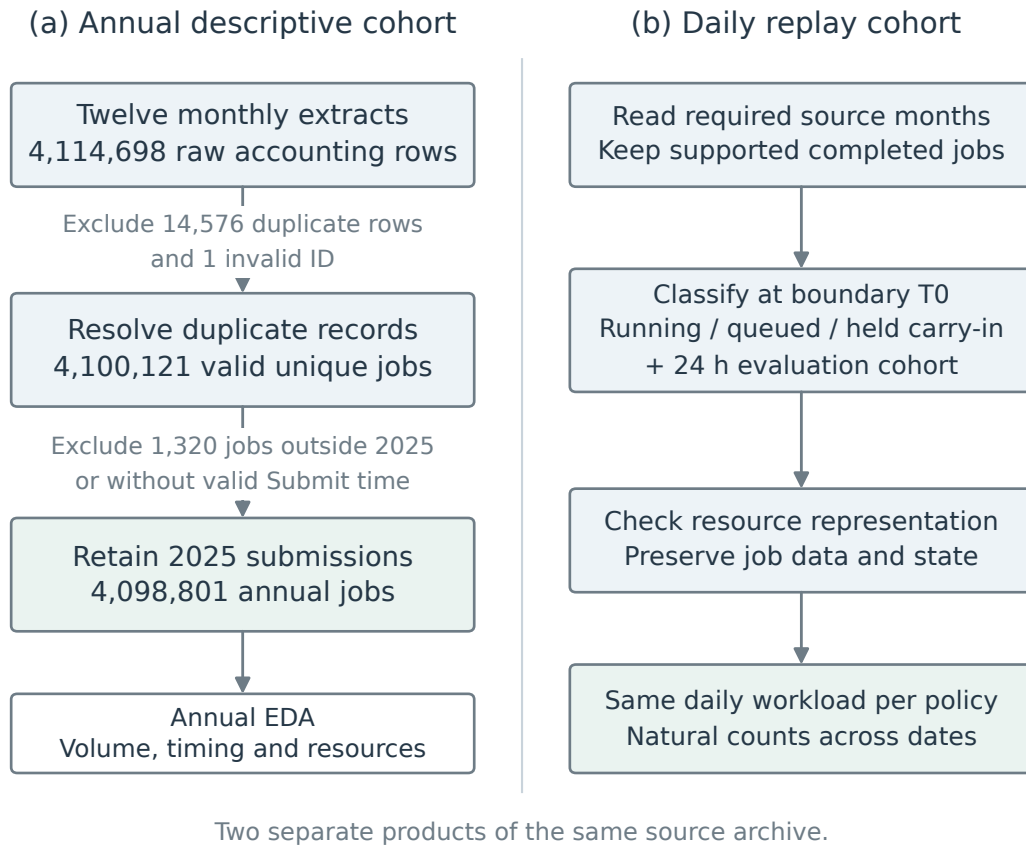


Figure 3.1: Source-data processing for annual description and daily replay. The left panel reports verified archive counts. The right panel describes the separate replay-construction rules and contains no policy outcome. Daily evaluation jobs and carry-in work are defined under the declared eligibility, validity and resource filters.

3.3 Exploratory data analysis

The EDA examines volume, job states, resource classes, runtime, waiting and arrival patterns before selecting evaluation workloads. Monthly variation is useful for identifying changes in the archive and interpreting later temporal checks. A large number of arrivals does not by itself demonstrate high electrical utilisation: a short CPU task and a long multi-GPU task contribute differently to occupied resources.

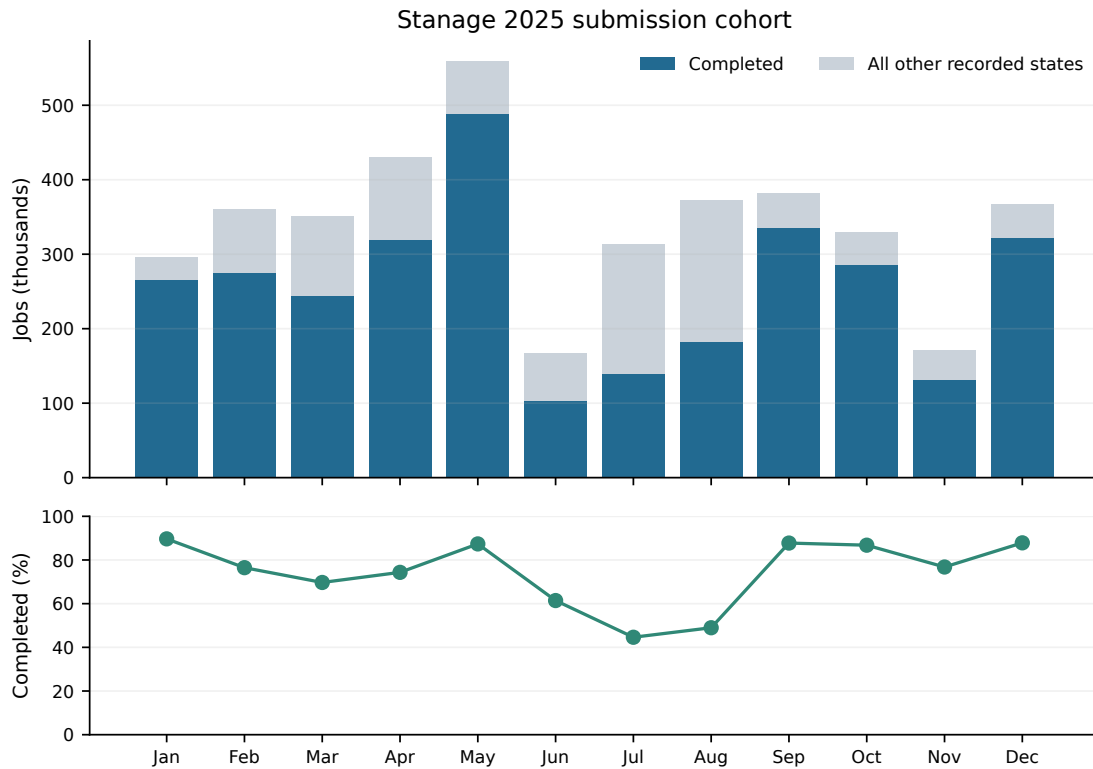


Figure 3.2: Monthly workload and recorded completion proportion in the 2025 submission cohort (4,098,801 retained jobs). Duplicate JobIDRaw records were resolved using the latest source month. All other states include failure, cancellation and non-completed states; the plot does not identify their causes.

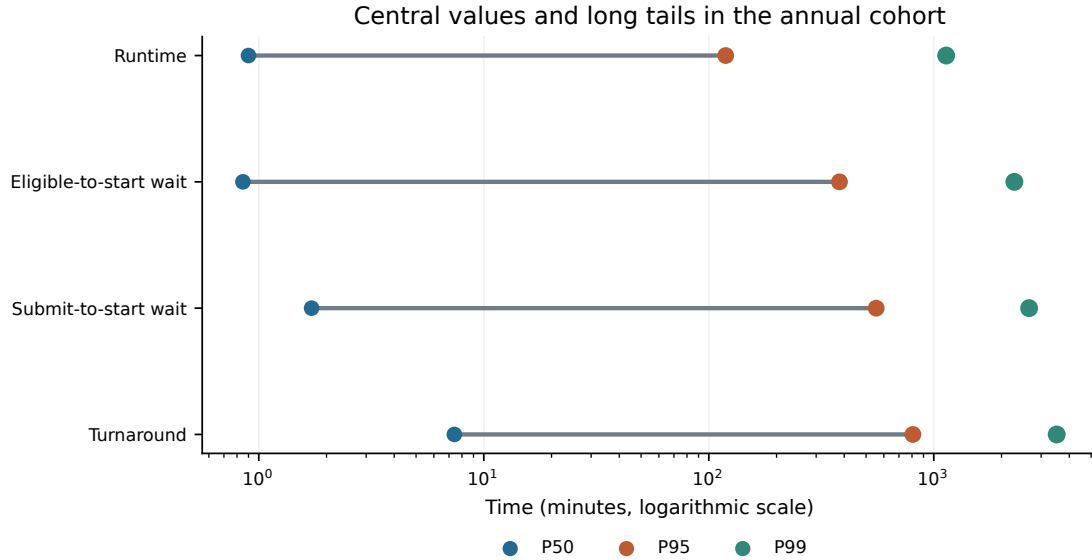


Figure 3.3: Annual timing percentiles from the retained 2025 cohort. Units are minutes, following the CSV unit column despite metric names ending in sec. Valid counts differ: runtime and submit wait 4,006,265; eligible wait 4,006,258; turnaround 4,098,801. Lines connect P50 and P95 only and are not confidence intervals.

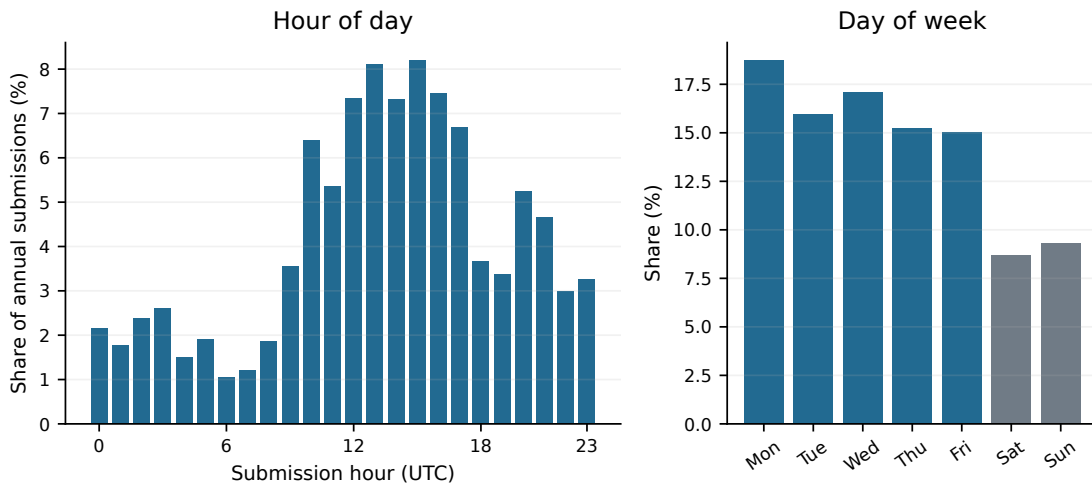


Figure 3.4: Submission-time distribution for 4,098,801 retained 2025 jobs, interpreted in UTC. Percentages are annual totals by clock hour and weekday, not per-day average arrival rates or measures of available scheduling flexibility.

The state audit is also part of interpretation. Low completion fractions in July and August are associated with failed records concentrated among a small number of pseudonymous users. They should not be explained as a holiday effect without independent evidence. Likewise, the absence of usable non-zero per-job energy fields limits the archive’s ability to validate electrical estimates, even though it provides substantial scheduling information.

Earlier June experiments used short pilot windows and a continuous daily window to develop the event pipeline and candidate policies. These remain development evidence because their results informed subsequent choices. The later 12 November comparison was initially frozen, but November then informed the development of the December low-impact rule. Its current role is consequently historical comparison and diagnosis. Describing it as the final untouched test of every later policy would misstate the sequence of work.

3.4 Daily cohorts and carry-in reconstruction

The current multi-day design uses 13, 15 and 18 November and 9 and 10 December 2025, each beginning at 00:00 UTC. Dates were selected from calendar and trace availability before inspection of the new policy outcomes. The set includes weekdays, a weekend date and two months. It is a designed case-study sample, not a random sample of the year.

For each date, supported completed jobs eligible in the following 24 hours form the evaluation cohort under the existing partition, resource and validity filters. The runtime cap is 96 hours. One evaluation job was shortened from 847,856 to 345,600 seconds by this cap, identically in every policy for its date. Completion therefore means completion of the configured workloads. A full-day replay covers the eligible population within the declared filters, not every production record. The work continues through queue drainage; it is not stopped after the 24-hour arrival period.

Three carry-in categories represent the pressure already present at the boundary. Running jobs are admitted first with their remaining runtime. Jobs already eligible but still queued are released at the boundary. Earlier-submitted held or dependency-bound jobs enter at their recorded later eligibility. Carry-in classification takes precedence if definitions overlap. Carry-in jobs cannot be delayed by a policy and do not enter the evaluation waiting percentiles, but their resource occupation contributes to energy and carbon accounting.

This reconstruction gives each policy the same non-empty starting conditions. It does not recover original priority ages, exact historical node placement, reservations or private fair-share state. The distinction between reconstructed state and reproduced production behaviour is assessed through replay diagnostics rather than hidden by the word “exact” in a script name.

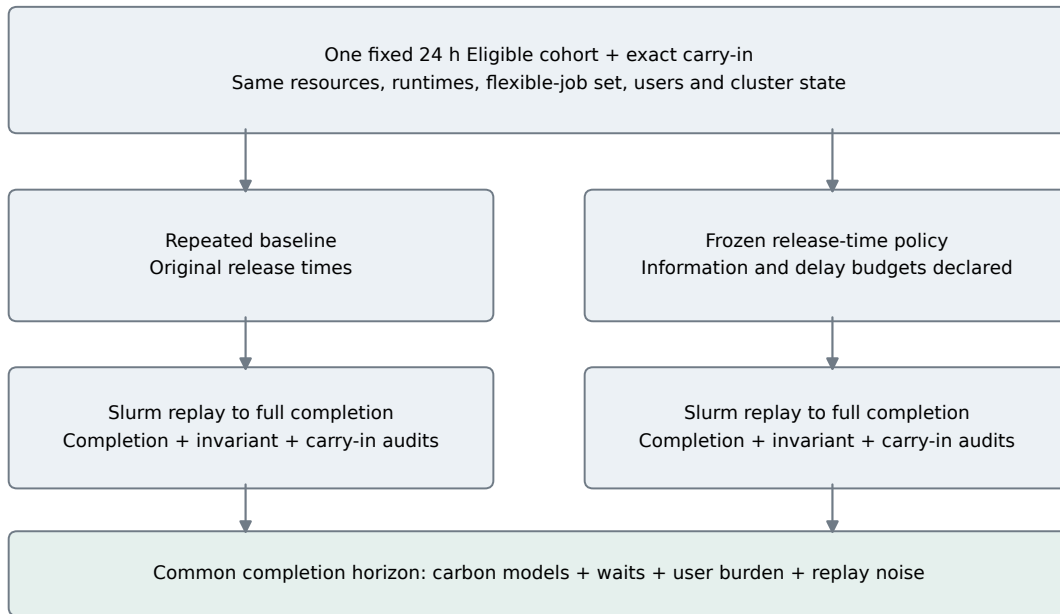


Figure 3.5: Paired replay design. The same date-specific cohort and initial state enter baseline and policy runs, which continue to completion and share a carbon-accounting horizon. The diagram describes the protocol; it does not imply that the five-date campaign has completed.

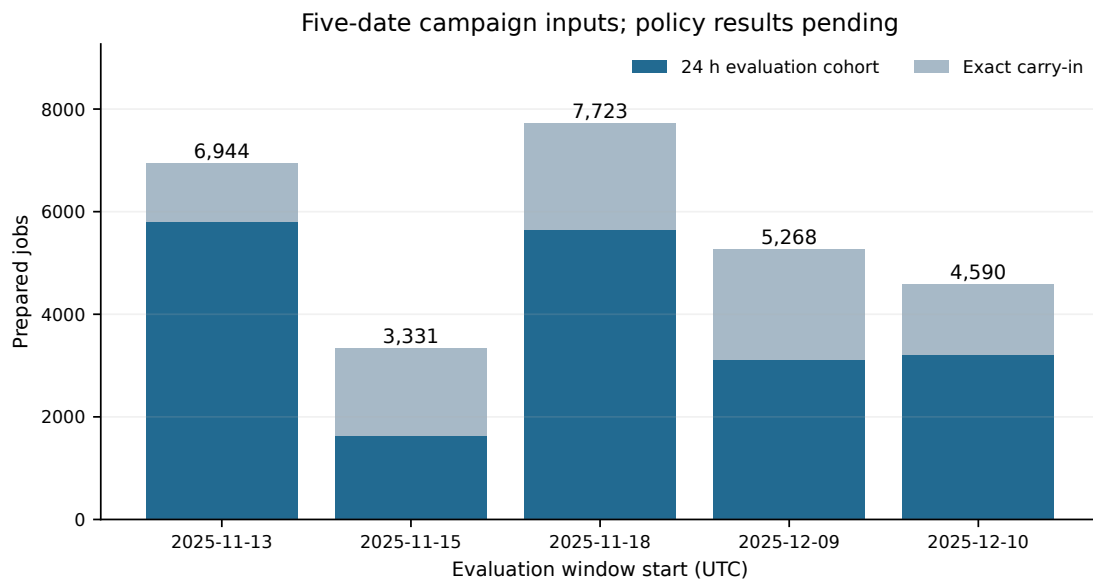


Figure 3.6: Prepared workloads for the five prespecified UTC dates. Each date retains its natural workload size, and all policies within the date receive the same evaluation and carry-in jobs. The figure describes inputs only; it contains no carbon outcome. Cross-date carry-in and completion tails can overlap, so summing these counts does not give unique annual jobs.

3.5 Simulated cluster and flexibility

The trace inventory contains 202 worker-node names, grouped into CPU, A100, H100 and H100-NVL classes for simulation. Public Stanage documentation supplies contextual hardware information [26]; mapping it onto the 2025 trace is a modelling step, because a current web page is not a timestamped historical inventory. Node and partition configuration are held fixed within the comparison.

A target probability of 30% assigns flexibility to eligible non-interactive jobs with a fixed seed of 42. The realised fraction can differ from exactly 30% on an individual date. The same assignment is reused for every policy on a date. This provides a controlled intervention population, but it does not identify actual user willingness to wait. Policy findings must therefore be qualified by the assumed flexibility fraction. No job is killed, checkpointed, migrated or given fewer resources to obtain a favourable result.

Temporal-shifting studies treat flexibility and interruption assumptions as determinants of the available opportunity [25, 29]. The fixed assignment here is an experimental control for that opportunity, rather than a forecast of adoption by real users.

3.6 Policies and information access

The main experiment compares seven representative policies against normal admission. B1 runtime shifting searches within four hours using carbon exposure across the given runtime. B2 delays flexible work when both carbon intensity and projected load are high. Adaptive Balanced combines runtime carbon, waiting and congestion with a runtime-proportional delay budget. A direct hybrid uses B2 during 08:00–18:00 UTC and runtime-aware B1 otherwise. The history-only and low-impact history-only variants estimate decision runtime and load from earlier months. Dynamic Forecast Balanced uses the completed baseline schedule as an oracle diagnostic.

For November, history-based decision tables use January–October; for December, they use January–November. This temporal restriction applies to predictors. The simulator still needs a runtime for execution, and the distinction is preserved in the workload and policy audit fields. The observed-runtime methods and oracle method are informative counterfactuals; they cannot be labelled as tested online predictors.

The representative oracle and history configurations also differ in minimum-saving thresholds, waiting/congestion weights and carbon-return gates. Their comparison therefore combines information access with policy safeguards; it is not a controlled ablation that identifies the causal effect of prediction error alone. A future information-only ablation would need to hold those settings constant.

All methods share the historical regional carbon curve. The saved series does not include the issue time of each forecast as it would have been known when a production decision was made. Therefore “history-only” denotes separation of runtime/load training information, not

the absence of hindsight throughout the entire experiment. Forecast-noise screens from earlier work are sensitivity tests, not measured forecast accuracy.

Table 3.2 collects the information conditions and key frozen parameters before the implementation details in Chapter 4. In particular, the oracle and history rows expose their different safeguards, so the information comparison can be read with those additional differences in view.

Table 3.2: Information conditions and selected frozen settings for the five-date design. All rows use the same retrospective regional carbon series and 30% target flexibility. R is given observed runtime; $\hat{R}$ is a history estimate. Conditions describe decisions, not measured outcomes.

Policy	Decision information	Delay bound	Key settings
B1 runtime	Given R ; runtime-average intensity	4 h	Minimum average intensity; no load penalty.
B2	Given R for projected node requests; release-time intensity	4 h	Carbon q75; node-request cap = 50% of high-carbon reference peak, minimum 1 node.
Adaptive	Given R ; CPU/GPU class-load projection	$\min(4 \text{ h}, R)$	2% intensity gate; wait/congestion weights 0.10/0.25; carbon q75; class threshold 0.50.
Day/night hybrid	Given R ; information follows selected branch	4 h	B2 at 08:00–18:00 UTC; runtime B1 otherwise.
History-only	Earlier-month runtime, load and queue-wait q75	$\min(4 \text{ h}, \hat{R})$	Capacity/node saving gates 2%/2%; weights 0.10/0.25; class threshold 0.75; return gate 0.05.
Balanced oracle	Given R and completed baseline schedule	$\min(4 \text{ h}, R)$	Capacity/node gates 1%/0.5%; weights 0.03/0.10; class threshold 0.75; return gate 0.02.
Low-impact history	Earlier-month predictions; runtime q25 for delay budget	60 min; also $0.5\hat{R}_{q25}$ and source time limit	Capacity gate 1%; weights 0.01/0.25; class threshold 0.90; return gate 0.001; per user: 60 min and 2 moved jobs.

Weights are wait/congestion coefficients. Return gates use the generator’s carbon-saving proxy per policy-delay hour; they are not validated electrical savings. History and oracle differ in parameters as well as information, so their comparison is not an information-only ablation. B2’s cap is a node-request admission threshold, whereas the other class thresholds enter congestion terms; none is a hardware power limit. Full configuration and prediction safeguards remain in the frozen manifest.

3.7 Operational carbon model

The primary model translates interval occupancy into a power scenario and integrates it with regional carbon intensity. For class h and half-hour interval k , CPU and GPU fractions are calculated from the occupied resource-seconds divided by configured capacity-seconds. The

class index is

$$U_{h,k} = \min\{1, \max(u_{h,k}^{\text{CPU}}, u_{h,k}^{\text{GPU}})\}, \quad U_k = \sum_h \frac{N_h}{202} U_{h,k}. \quad (3.1)$$

The GPU fraction is zero for CPU-only classes. N_h is the number of nodes in the class. This index measures normalised allocation, not processor activity or electrical power. Equal node weights are a transparent simplification rather than a claim that CPU and GPU nodes draw equal power.

Approximate whole-cage values supplied in supervision define

$$P_k = 140 + 55U_k \quad \text{kW}. \quad (3.2)$$

The 140 kW term is the fixed scenario baseline; 195 kW corresponds to $U_k = 1$. These inputs cover equipment inside the cage, including compute, storage and networking. Facility overhead outside that boundary has not been quantified, so no additional PUE multiplier is applied. The general energy-to-carbon conversion follows established accounting practice: Green Algorithms gives the energy and carbon relations on PDF pages 7–8 [11], and Carbontracker gives interval energy and carbon relations on PDF page 8 [1]. The coefficients and occupancy mapping above are project assumptions.

For intensity I_k in grams of CO₂ per kWh and duration Δt_k in hours, the load-dependent and whole-cage estimates are

$$C_{\text{load}} = \frac{1}{1000} \sum_k 55U_k \Delta t_k I_k, \quad (3.3)$$

$$C_{\text{cage}} = \frac{1}{1000} \sum_k (140 + 55U_k) \Delta t_k I_k. \quad (3.4)$$

Both return kilograms of CO₂. NESO describes its electricity-intensity signal as generation CO₂ intensity [16, 17]. Legacy project columns containing `co2e` are retained in source provenance, but this dissertation does not turn that signal into a measured full greenhouse-gas lifecycle footprint.

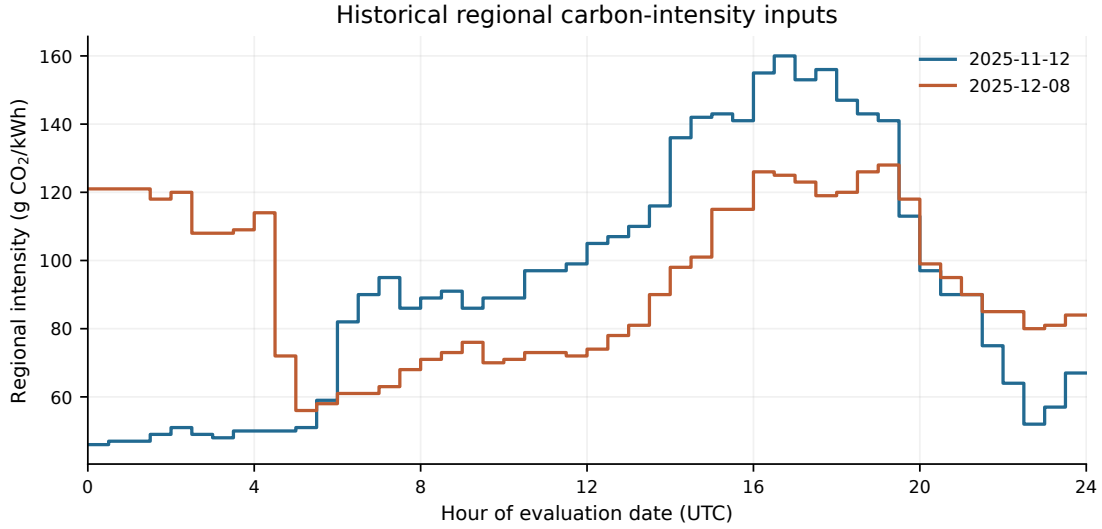


Figure 3.7: NESO regional historical modelled carbon-intensity series used for the November and December case studies, showing the first 24 h of each evaluation window. Each step is 30 minutes. Full emissions accounting includes each common completion tail. The archive has no matched as-issued forecast timestamps and final actual series, so this is a retrospective carbon-signal input, not demonstrated forecast accuracy. The source represents electricity-generation CO₂, not lifecycle CO₂e.

Two sensitivity indices are reported: capped requested-node occupancy and distinct allocated-node occupancy. Requested nodes can overcount shared nodes before capping; distinct allocation merges overlapping intervals on each host. The latter avoids that counting artefact but remains sensitive to simulated node placement.

Slurm node-sharing studies show why placement and sharing configuration can change the relationship between job requests and occupied hosts [21]. The three indices consequently answer related allocation questions, rather than providing three independent electrical measurements. Reporting both checks whether a favourable result depends on one allocation representation.

This is a sensitivity analysis of a power-model input, consistent with the dependence on hardware and workload characteristics described in the HPC modelling literature [18]. It does not estimate a confidence interval for measured power, because no independent power series is available for that calibration.

Figure 3.8 shows how the three occupancy representations enter the same accounting calculation. Keeping the physical boundary and the fixed component visible helps distinguish a load-related saving from a whole-cage percentage.

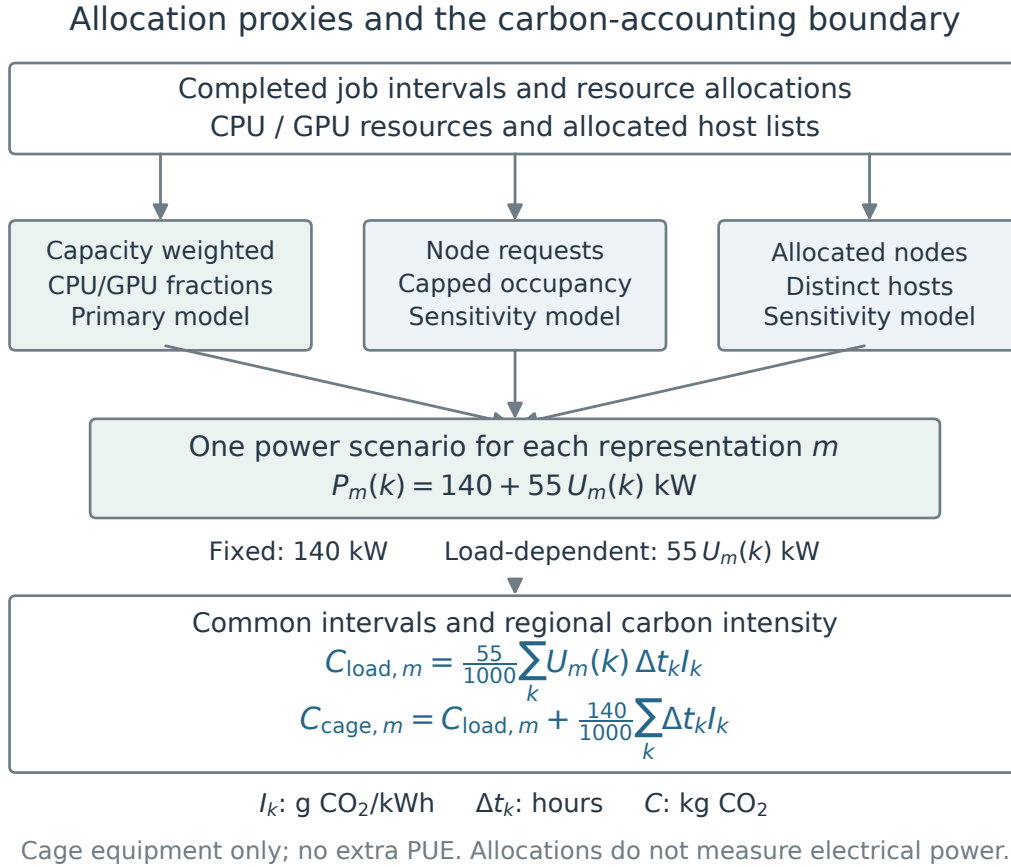


Figure 3.8: Conversion from job allocations to modelled operational CO₂. The primary capacity representation and two node-based sensitivity representations use the same linear cage-power scenario, regional intensity and completion horizon. Fixed and load-dependent components remain separate. Allocation-based occupancy is not measured processor activity or power; the regional signal represents electricity-generation CO₂.

Final accounting uses the same interval for every required replay of a date: from the date boundary to the latest modelled completion, rounded to the carbon interval grid. The accounting start is the release time plus the non-negative logged scheduler wait; the modelled end is this start plus the configured runtime. It is not the potentially late completion timestamp in the event log. This distinction matters to the timing audit in Section 5.5. The fixed 140 kW integral cancels in a paired absolute carbon difference, but remains in the denominator of a whole-cage percentage. A load-dependent percentage reduction is therefore not the same percentage reduction in total cage emissions.

Table 3.3: Symbols and units in the carbon calculation.

Symbol	Unit	Meaning
h, k	–	Resource class and half-hour accounting interval.
N_h	nodes	Number of simulated nodes in class h ; total 202.
$u_{h,k}^{\text{CPU}}, u_{h,k}^{\text{GPU}}$	–	Occupied resource-seconds divided by class capacity-seconds.
$U_{h,k}, U_k$	–	Class occupancy index and node-weighted cluster index, each bounded by zero and one.
P_k	kW	Scenario power, not a job-level measurement.
Δt_k	h	Duration accounted for in interval k .
I_k	g CO ₂ /kWh	Regional electricity-generation carbon intensity.
$C_{\text{load}}, C_{\text{cage}}$	kg CO ₂	Load-dependent and fixed-plus-load carbon estimates.

3.8 Service metrics and user burden

For job j , let s_j , e_j and r_j denote source submission, eligibility and intended policy release. Let ℓ_j and b_j denote logged simulator submission and start, expressed relative to the same time origin. The reported components of waiting are

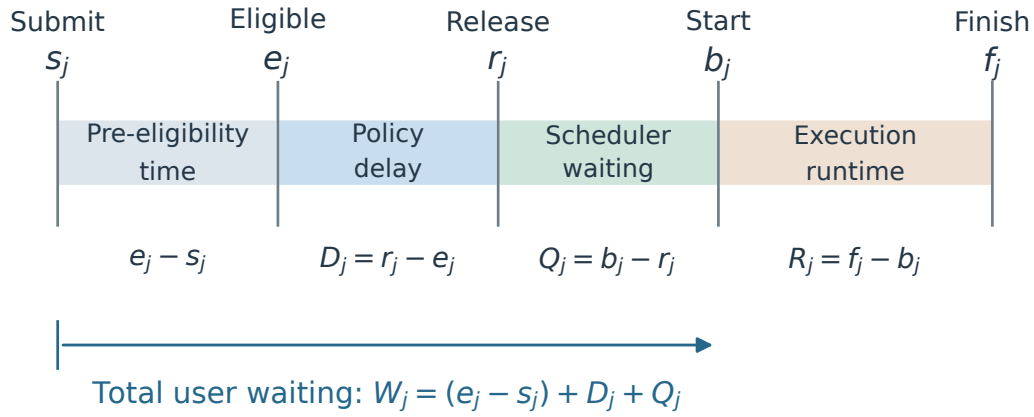
$$D_j = r_j - e_j, \quad Q_j = b_j - \ell_j, \quad (3.5)$$

$$W_j = (e_j - s_j) + D_j + Q_j. \quad (3.6)$$

D_j is deliberate policy delay, Q_j is logged scheduler waiting and W_j is the reported total user wait. Intended release and logged submission can differ; the calculation combines the intended hold with logged queue waiting, rather than assuming $r_j = \ell_j$. Figure 3.9 shows the idealised components without this submission-timing discrepancy. Moving one job can reorder the queue and reduce a population percentile while increasing another job’s wait.

Figure 3.9 makes these components explicit on a symbolic job timeline. In particular, an admission delay and a change in total waiting are different quantities even when they are expressed in the same unit.

One evaluation job: event order and waiting components



Schematic only: distances do not represent measured durations.

Figure 3.9: Idealised waiting components, assuming intended release and logged Slurm submission coincide. Submission-to-eligibility precedes admission; policy delay ends at release; scheduler waiting ends at start. The reported metric uses logged submission for scheduler waiting when these boundaries differ. The timeline is symbolic and not to scale.

Waiting percentiles, maxima, turnaround and slowdown use the same evaluation cohort. The existing throughput field instead divides all completed replay jobs, including carry-in, by the full replay span; it is reported with that denominator rather than called evaluation-only throughput. The implemented slowdown indicator is $(R_j + W_j)/\max(R_j, 60\text{ s})$. Unlike definitions that additionally clamp the ratio to one, this implementation may produce values below one for short jobs; the equation states exactly what is computed. User-level reporting includes cumulative policy delay, the number of affected jobs, maximum burden and the largest user’s share of total policy delay. The count delayed longer than execution runtime is reported separately from compliance with a predictor-based budget.

For a policy p and baseline 0, carbon reduction is

$$S_p = 100 \left(1 - \frac{C_p}{C_0} \right) \%. \quad (3.7)$$

Positive values mean reduction and negative values mean increase. Carbon saved per cumulative policy-wait hour can support efficiency comparisons, but is undefined when no delay is imposed and cannot replace the absolute reduction and burden distributions.

3.9 Repetition and multi-day interpretation

Each date received three baseline runs, seven policy runs and one additional Adaptive Balanced run, giving 55 final completed replays. The original plan interleaved baselines with a rotated policy sequence. A recorded execution amendment increased concurrency to five isolated containers across two virtual machines, using the same pinned image and clock scale of 100. The additional environment controls were diagnostic runs outside the 55. Dates, job populations, policy parameters and acceptance rules did not change. Controls on 13 November supported the amended execution on that configuration; they do not establish negligible variation on every date.

Baseline medians define the primary paired carbon reference, and the median of two replays represents Adaptive. Ranges describe observed repeat variation. Other policies have one replay per date, so five dates are not five repeats of the same scenario. Their run-to-run variability is not fully characterised by the available baseline repeats.

Table 3.4: Five-date workload inventory before policy comparison. GPU jobs are a subset of each total. Per-date counts may overlap through carry-in and are not unique annual job totals.

Date (UTC)	Evaluation	Carry-in	Total	GPU jobs
2025-11-13	5,806	1,138	6,944	226
2025-11-15	1,629	1,702	3,331	486
2025-11-18	5,646	2,077	7,723	387
2025-12-09	3,113	2,155	5,268	774
2025-12-10	3,210	1,380	4,590	727

The allocation of repeated and single runs is shown in Figure 3.10. All planned cells were completed. Failed or superseded attempts remain in the audit archive and are not counted as extra policy observations.

Five-date experiment design: planned replays per cell

	Baseline	B1	B2	Adaptive	Hybrid	History	Oracle	Low-impact
13 Nov 2025	3	1	1	2	1	1	1	1
15 Nov 2025	3	1	1	2	1	1	1	1
18 Nov 2025	3	1	1	2	1	1	1	1
09 Dec 2025	3	1	1	2	1	1	1	1
10 Dec 2025	3	1	1	2	1	1	1	1

5 dates × 11 runs per date = 55 planned full replays

3 baseline repetitions; 2 Adaptive repetitions; 1 for each other policy.

Matched workload and initial state within each date; common accounting horizon.

Design only: cells are planned counts, not completed runs or carbon outcomes.

Figure 3.10: Prespecified replay allocation across five dates, completed in the final evidence set. Cell values are repetition counts: three baseline runs, two Adaptive runs and one run for each other policy, totalling 55. The same workload is used within each date, while natural workload sizes differ across dates. The matrix describes allocation rather than carbon outcomes; archived failed attempts and environment controls are not extra policy observations.

The frozen working screen requires five valid dates, material positive primary-model change on at least four, and a worst-day increase no greater than 0.1%. A material positive day must strictly exceed the maximum of 0.05%, the baseline repeat range and the available policy repeat range, and pass the conservative minimum-baseline versus maximum-policy comparison. Both repeat ranges are expressed as percentages of median baseline carbon. The screen also requires positive baseline-carbon-weighted aggregate reduction and positive mean direction when any one date is left out. These are project screening rules, not operational standards or significance tests. Their thresholds were not relaxed after inspecting the results.

Service impact is assessed separately. A lower-impact date requires a P95 increase of at most five minutes, no policy delay longer than configured runtime, at most 60 cumulative delay minutes and two delayed jobs per user, and a largest-user burden share of at most 40%. Carbon stability does not replace these service measures.

Different dates retain their natural job counts. Equal-day means, medians and worst dates summarise the five cases; a separate pooled percentage uses the sum of baseline carbon as its denominator. An average of daily P95 values is an average of percentiles, not a pooled job-level P95. Carry-in and execution tails can overlap, so the five dates are not independent random observations or disjoint windows for annualising savings.

The user-centred summary first normalises each day's shared carbon difference by its evaluation-job count, then weights dates equally:

$$g_{p,d} = \frac{1000(C_{0,d} - C_{p,d})}{N_d}, \quad (3.8)$$

$$w_{p,d} = \text{median}_{a,b} \left[\frac{1}{N_d} \sum_{j \in J_d} \left(W_{p,d,j}^{(a)} - W_{0,d,j}^{(b)} \right) \right], \quad (3.9)$$

$$\bar{g}_p = \frac{1}{5} \sum_d g_{p,d}, \quad \bar{w}_p = \frac{1}{5} \sum_d w_{p,d}. \quad (3.10)$$

Here p is a policy, d a date, J_d its evaluation cohort and $N_d = |J_d|$ its size. $C_{0,d}$ and $C_{p,d}$ are median shared-queue carbon estimates in kg CO₂; a and b index policy and baseline repeats. $W_{p,d,j}^{(a)}$ is job j 's total waiting in seconds. Thus $g_{p,d}$ has units of g CO₂ per evaluation job and $w_{p,d}$ has units of seconds per evaluation job. Carbon includes carry-in and queue-wide effects: $g_{p,d}$ is a scale-normalised cluster difference, not an individual job footprint. Wait differences are calculated for matching job IDs within each replay pair before taking the median pair statistic. Positive-only added wait replaces each difference inside the sum by its maximum with zero, exposing harm that earlier starts elsewhere could otherwise offset.

3.10 Validity and completion requirements

Workload identity, resource invariants, carry-in membership, permitted release changes, complete carbon coverage and terminal job completion are checked before a replay enters the comparison. An incomplete date remains visible rather than becoming zero reduction or disappearing from the five-date denominator. Carbon coverage is provisioned for 14 days; insufficient coverage causes an error rather than interpolation or zero filling.

The frozen manifest and retained deviations support the distinction between planned analysis and later exploration encouraged in research-computing guidance [24]. This is a documented local plan, not an externally registered preregistration. Additional tuning after inspecting the new outcomes would need to be identified as a subsequent development stage.

Chapter 6 assesses the remaining modelling and generalisation limits.

Chapter 4

Design and Implementation

4.1 Pipeline architecture

The pipeline separates data preparation, admission decisions, simulation and evaluation. This makes the inputs to a decision inspectable and allows the same completed replay to be analysed under more than one occupancy model. The main artefacts are a workload profile, simulator configuration, event stream, policy decision audit, terminal job log and interval/result summaries. A manifest records parameters and content hashes so that a policy label alone is not treated as a complete experiment description.

The implementation grew from small simulator examples into a trace-based workflow. Early synthetic workloads were useful for understanding events, virtual users, partitions and multi-node requests. They are implementation tests rather than evidence about Stanage carbon reductions. The dissertation’s empirical comparisons use trace-derived workloads and preserve the distinction between small fixtures and completed experimental runs.

Figure 4.1 locates the policy generator relative to the simulator and accounting stages. The separate oracle return path identifies where completed-baseline information enters that diagnostic variant.

4.2 Trace conversion and event generation

The workload generator reads the relevant archive period and constructs a profile with source timing, resource class, requested CPU/GPU/node resources, a pseudonymous user and the execution runtime. The profile also retains source time limits and explicit UTC epoch information. Event offsets are interpreted relative to this epoch, avoiding ambiguous alignment between simulated time, carbon intervals and original timestamps.

Admission decisions update the release offset. Resources, identity and runtime remain unchanged. The resulting events are regenerated in chronological order and supplied to the simulator alongside `slurm.conf`, `sim.conf`, `gres.conf` and virtual-user definitions. A comparison can therefore check the entire profile rather than only a job-count total: two workloads with equal counts may still differ in resources or membership.

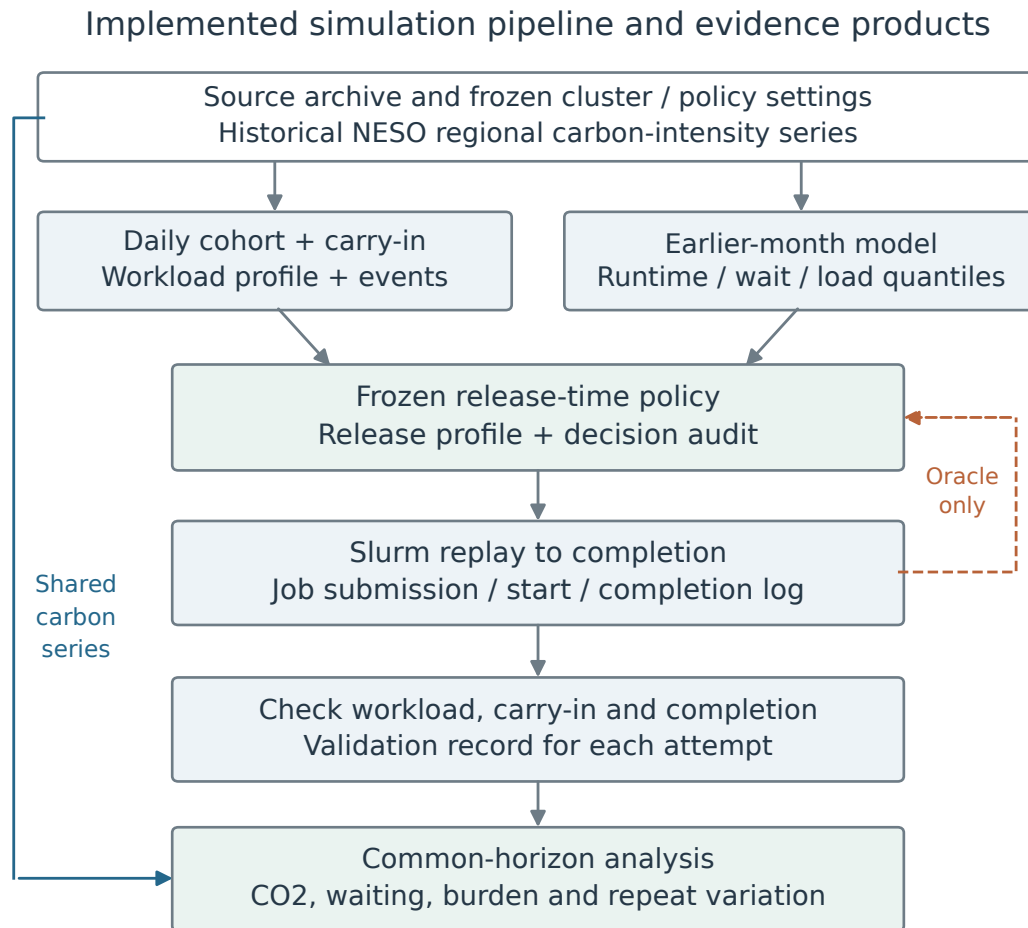


Figure 4.1: Data flow through the implemented simulation pipeline. Earlier-month aggregates supply history predictors. The oracle branch additionally reads a completed baseline schedule, shown by the dashed return path. Policy profiles enter Slurm, while recorded execution and validation outputs support common-horizon accounting. This is an experimental workflow, not a deployed production controller.

The carry-in builder identifies the categories defined in Chapter 3. Running work has admission precedence and receives the remaining execution duration. Queued work enters at the evaluation boundary. Held work retains its later eligibility. The carry-in audit checks category membership, release offsets, absence of policy delay and completion. It also measures running-job startup tolerance, since a simulated release at the boundary does not guarantee an identical historical allocation immediately afterwards.

4.3 Slurm integration

Slurm backfill and resource allocation determine actual starts after admission [20]. The intervention does not modify production priority weights, fair-share rules or preemption behaviour. A job admitted during a low-intensity interval can still start later if resources are unavailable. Consequently, the implementation records both planned release and realised simulated start; policy-generation estimates are not substituted for post-replay outcomes.

Local development included correcting a seconds-to-microseconds conversion in the scheduler source and stabilising the simulator clock scale at 100. Runs are executed with a fixed container image. The runner inspects unique submission, start and completion events and checks that no expected work times out. Some earlier runs terminated abnormally after the final completion event, which motivated checking event completeness in addition to process status. Neither a zero process exit nor a final log line alone establishes a valid result. The relevant run diagnostics are retained with the evidence.

The simulator exercises real scheduling code but does not execute the scientific applications themselves. It therefore represents scheduling and allocation interactions without measuring application activity, GPU kernels or node electrical demand. This is the reason energy analysis remains a separate scenario model.

4.4 Admission rules

The three given-runtime rules offer a direct mechanism comparison in Figure 4.2. They receive the same type of job and historical carbon input, but differ in which release conditions they evaluate and how they limit waiting. The common lower stage is Slurm replay, where the realised start and allocation are determined.

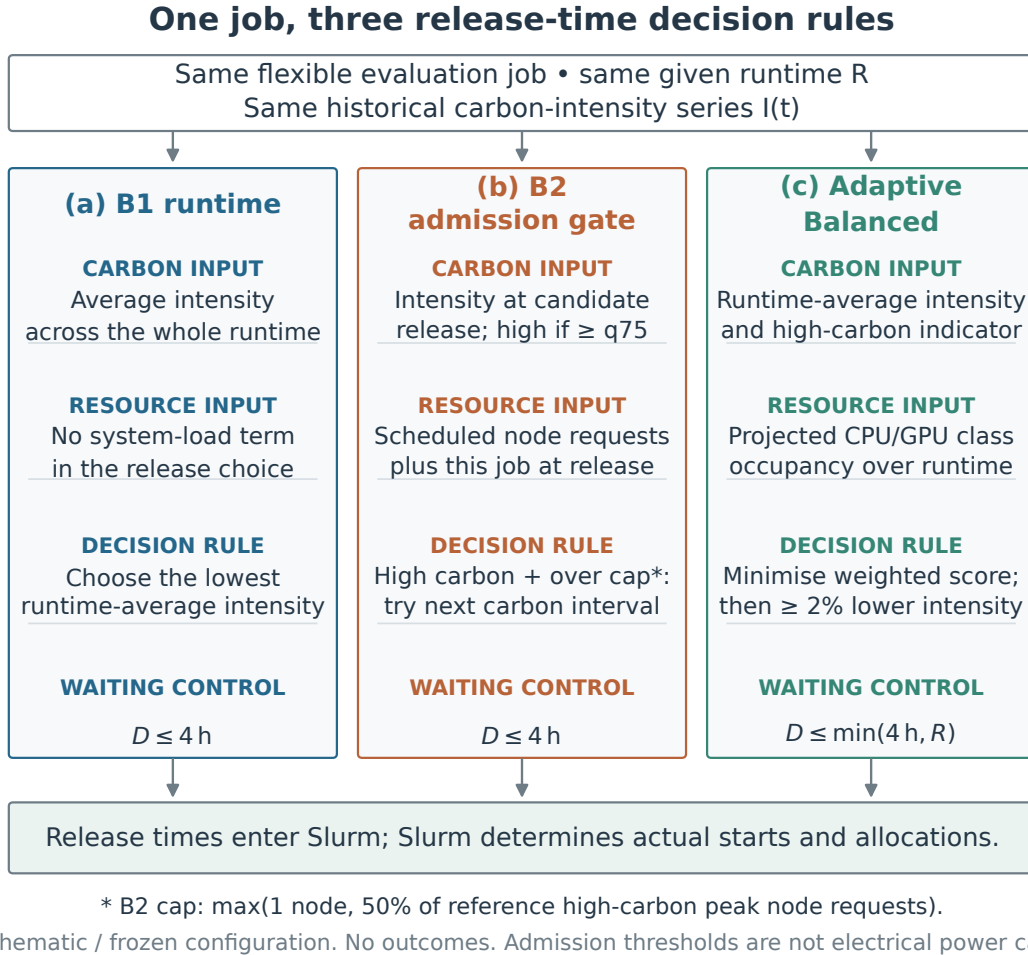


Figure 4.2: Comparison of three frozen release rules for the same flexible evaluation job. B1 uses runtime-average carbon intensity; B2 uses instantaneous intensity and node-request concurrency; Adaptive adds waiting and class-load penalties to runtime carbon. Adaptive weights are 0.10 for waiting and 0.25 for congestion, with class threshold 0.50. These methods use given runtime and retrospective carbon information. The diagram is schematic and reports no experiment outcome; oracle and history-based variants have separate information and parameter settings.

4.4.1 Normal admission and runtime-aware shifting

Normal admission releases an evaluation job at its original eligibility. Runtime-aware B1 searches the current release and later half-hour candidates within a four-hour delay limit. For candidate t and decision runtime $\widehat{R}_j$, it estimates the time-weighted intensity

$$\bar{I}_j(t) = \frac{1}{\widehat{R}_j} \int_t^{t+\widehat{R}_j} I(\tau) d\tau. \quad (4.1)$$

Using the complete interval avoids treating the first low-intensity half-hour as representative of a long job. B1 uses given observed runtime in the current comparison. It is a controlled full-information reference for the consequences of shifting, rather than evidence that an online runtime predictor is accurate.

The interval objective follows the non-interruptible workload perspective used in temporal-shifting research [29]. The four-hour bound, half-hour candidate grid and observed-runtime input identify this project’s particular comparator; the citation does not supply those parameter values.

4.4.2 High-carbon admission gate

B2 identifies high-carbon intervals using the 75th percentile of the available intensity horizon. Its load threshold is 50% of the peak projected node requests in the reference release profile during high-carbon intervals, with a one-node minimum. This projection uses release offsets and given runtimes, rather than the completed Slurm baseline schedule used by the oracle method. Flexible work may be postponed to subsequent half-hour boundaries, with a four-hour limit. The rule does not replace Slurm’s resource feasibility checks.

The word “cap” refers to an admission/concurrency threshold. It does not mean that hardware electrical power is constrained to a measured number of kilowatts. Resource occupancy can change while the fixed cage-power component remains unchanged. This distinction is required when interpreting B2 relative to hardware power management described in the literature.

For example, Zeus explicitly changes GPU power limits during DNN training [32]. B2 makes an admission decision and leaves that hardware-control variable untouched, so its threshold must be interpreted in the units of projected node requests.

4.4.3 Adaptive Balanced

Adaptive Balanced evaluates the same carbon opportunity with explicit waiting and congestion costs. Its delay budget is

$$B_j = \min(D_{\max}, \rho \widehat{R}_j), \quad (4.2)$$

where $D_{\max} = 4$ hours and $\rho = 1$. For each candidate within that budget, the score is

$$F_j(t) = \frac{\bar{I}_j(t)}{\bar{I}_j(e_j)} + \lambda_w \frac{t - e_j}{D_{\max}} + \lambda_c G_j(t). \quad (4.3)$$

$G_j(t)$ is the projected high-carbon congestion penalty: it uses positive class-load excess above the capacity setting in high-carbon intervals and is zero otherwise. The frozen weights are $\lambda_w = 0.10$ and $\lambda_c = 0.25$. A move must also reach a minimum expected runtime-average intensity improvement of 2%. The high-carbon class-capacity setting is 50%.

Jobs are processed in eligibility order. Non-flexible and carry-in work retain their release times. For a flexible job, the generator enumerates candidates, evaluates interval carbon and projected congestion, and selects the lowest permitted score. An accepted interval updates the projected load used for subsequent decisions. The job audit preserves the budget, selected delay and decision reason.

Figure 4.3 gives the implementation order: select the candidate with the smallest score, then apply the minimum carbon-improvement gate. If that selected candidate fails the gate, the job keeps its original release. The expected improvement remains a decision quantity until Slurm replay produces the realised schedule.

The policy expresses a preference for lower carbon while charging for delay and anticipated congestion. It is greedy and does not establish a global optimum over all possible job orderings. Its observed-runtime input also remains an information advantage. The multi-day comparison examines whether the explicit waiting constraint changes the empirical trade-off relative to B1 and B2.

The budget is motivated by the job-sensitive waiting control in Starburst, while the joint consideration of carbon and resource pressure is informed by GREEN [15, 30]. Neither source specifies this score or its frozen weights. The equations above define the implemented rule, and its utility remains an empirical question.

Adaptive Balanced: one job in eligibility order

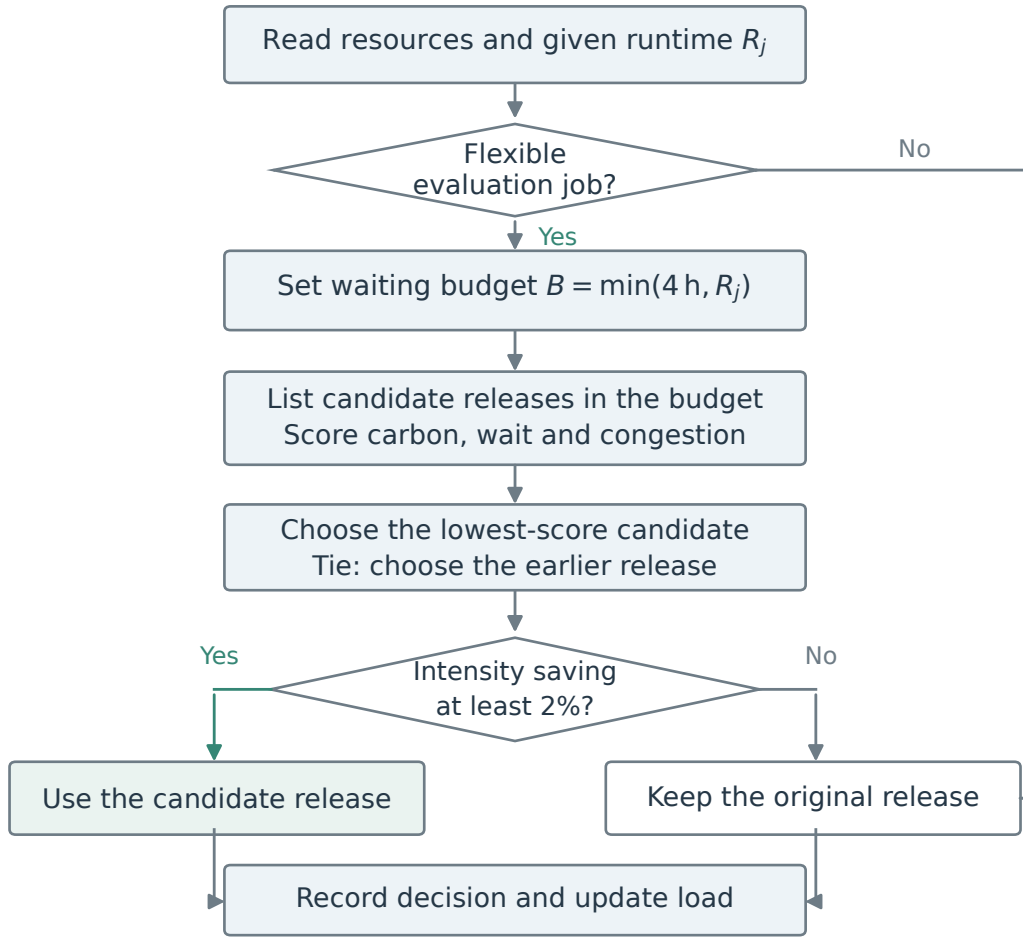


Figure 4.3: Decision sequence for the frozen Adaptive Balanced configuration. A given-runtime budget limits release candidates. The generator first selects the minimum score and then checks the required 2% runtime-average intensity improvement; otherwise it retains the original release. The congestion term is a score penalty, not a measured power cap. Given runtime remains an information advantage in this retrospective comparison.

4.4.4 Direct day/night hybrid

The hybrid follows the job’s UTC eligibility hour. Work eligible from 08:00 to 18:00 uses B2; the remaining hours use runtime-aware B1. Both branches have four-hour limits. The branch is recorded explicitly in the workload profile. This implements the supervisor discussion as a testable comparator. It does not assume that clock time alone consistently identifies low-carbon or low-congestion conditions.

4.4.5 Oracle load information

Dynamic Forecast Balanced reads a completed normal-admission schedule and forms interval-load arrays for CPU, A100, H100, H100-NVL and requested nodes. Before evaluating a job, its current contribution is removed. Candidate contributions are added under the same capping order used by the carbon estimator. An accepted move updates the arrays before later decisions.

The gate requires improvement in both primary capacity and node-request estimates and a sufficient return per unit of waiting. For the current comparison, the frozen minimum predicted savings are 1% and 0.5%, respectively, with a minimum carbon-return parameter of 0.02. Its four-hour/global and one-runtime/job limits remain explicit. These settings identify the saved configuration; they are not claimed to be universally optimal values.

The completed baseline contains future same-workload information. This method is therefore labelled “oracle” wherever results are compared. It helps diagnose whether more accurate load information changes release decisions and the carbon/wait trade-off. It is not a measured forecast service and does not invoke an AI agent.

4.4.6 History-only decisions

The history model builds runtime, queue-wait and interval-load quantiles from months earlier than the evaluation month. Runtime groups use resource characteristics and deterministic fallback levels where support is insufficient. Earlier development checked expanding history, recent-window history and prior-month calibration. Calibration is assessed on later observations separately from the data used to build the predictor.

The history-only policy obtains its decision runtime and load from frozen quantiles. It uses the dynamic marginal calculation without reading the evaluation job’s completed runtime or the future baseline schedule for the decision. The current configuration uses Q75 runtime, load and queue-wait estimates with a four-hour delay bound. This separation permits comparison of a history-informed policy on the same replayed work. Because the representative oracle and history rules also use different safeguards, observed differences cannot be attributed solely to information uncertainty.

The low-impact variant forms candidates from a five-minute grid, carbon-interval boundaries and the exact budget endpoint, up to 60 minutes. The endpoint can therefore produce a delay shorter than five minutes. It uses Q90 runtime for carbon exposure and a more

conservative delay allowance,

$$B_j = \min\{60 \text{ min}, 0.5\hat{R}_{j,25}, L_j, \text{remaining user delay budget}\}. \quad (4.4)$$

Here L_j is the non-negative declared source time limit; a zero value permits no additional delay. It is an extra duration bound, not a user completion deadline. A user can receive at most 60 minutes of cumulative policy delay and two delayed jobs in the window. Uncertain or unsupported long-GPU cases can retain baseline admission. Carbon, congestion and return thresholds are frozen in the campaign manifest. These constraints limit direct intervention but do not guarantee that Slurm’s subsequent queue response improves aggregate carbon or benefits every user.

4.5 Accounting and audit outputs

The analyser joins source profiles to simulator logs and separates dependency time, deliberate policy delay and scheduler wait. It retains allocated node lists for the distinct-node sensitivity calculation. The carbon calculator computes exact overlaps between job intervals and half-hour intensity bins, converts the occupancy index using Equations 3.2–3.4, and produces common-horizon scenario summaries.

Decision-time expectations and realised replay outcomes remain in separate fields. A predicted saving can disappear when later scheduling moves other jobs, or change sign under a different occupancy proxy. This separation is necessary for the information question: accurate arithmetic in a policy generator is not itself validation of its queue forecast.

Automated checks cover parsing, timestamp conversion, interval overlaps, release constraints, carry-in categories and complete replay denominators. Small test fixtures verify the code and are excluded from the main five-date results. A separately labelled V2 fixture illustrates controller behaviour. The writing package keeps aggregate figure sources and provenance while the full experiment directory retains detailed logs and profiles.

4.6 Multi-day execution

The campaign runner creates a frozen workload and associated policies for each date and saves progress after each complete attempt. The amended dispatcher runs up to five isolated scenarios concurrently, subject to baseline and oracle dependencies. It records input hashes, the image identifier, configuration, execution backend and audit results. Technical attempts are limited and retained. Resume logic skips only completed runs whose inputs and checks still match, preventing a partial directory from being mistaken for a finished experiment.

Once all relevant runs for a date are available, the summariser uses a shared model-completion horizon and repeated-baseline reference. It reports each date, then the distribution across dates. This ordering prevents a favourable pooled total from obscuring a difficult date. The final evidence set contains all 55 required replays and does not substitute zero for an incomplete attempt.

4.7 Engineering extension after the frozen comparison

The five-date findings prompted a separate revision, called V2 in the implementation. It was developed after the main comparison and did not replace its seven policies or their saved outputs. The purpose was to address identifiable decision and reporting weaknesses within a three-hour total test budget, not to search for another favourable parameter setting. Table 4.1 links the changes to the problems they address.

Table 4.1: Implemented improvements and the evidence required to assess them.

Change	Previous limitation	Implemented response
Feasibility before ranking	The best-scoring candidate could fail a gate while another candidate was feasible.	Filter candidate constraints first; rank eligible alternatives; retain immediate admission as a competitor.
Decision audit	A zero-action result did not always explain the reason.	Record candidate scores, rejection reasons, accepted actions and budget decisions; mark unrecoverable historical reasons as unknown.
Validation by category	Completion could be mistaken for timing or model validity.	Report completion, input identity, initial state, timing, carbon coverage and repeatability separately.
Queue feedback	A static hold did not react to the actual queue.	Reconsider unsubmitted held work on state changes and timer events; release it if the expected benefit disappears.
Shared user budget	Per-job bounds still allowed concentrated cumulative delay.	Count spent and reserved delay against a per-user limit and consider cheaper feasible alternatives.
Failure handling	Controller failure could strand work or reset the test budget.	Fail open by submitting work; persist one total deadline and a closed-experiment record.

4.7.1 Candidate selection and budget accounting

The frozen Adaptive implementation first chooses a score winner and then applies its carbon gate, as Figure 4.3 records. This is retained as the tested original behaviour. The independent V2 copy checks feasibility before ranking candidates in the affected Adaptive, history, forecast and marginal branches. A regression fixture demonstrates a case where the original rule abstains because its score winner fails the improvement gate, while a feasible alternative beats immediate admission. This establishes a code-level difference; it supplies no replacement carbon result for the frozen campaign.

The feedback controller limits each proposed direct hold to the smaller of 30 minutes and half the historical predicted runtime. Each user also has a cumulative 30-minute budget.

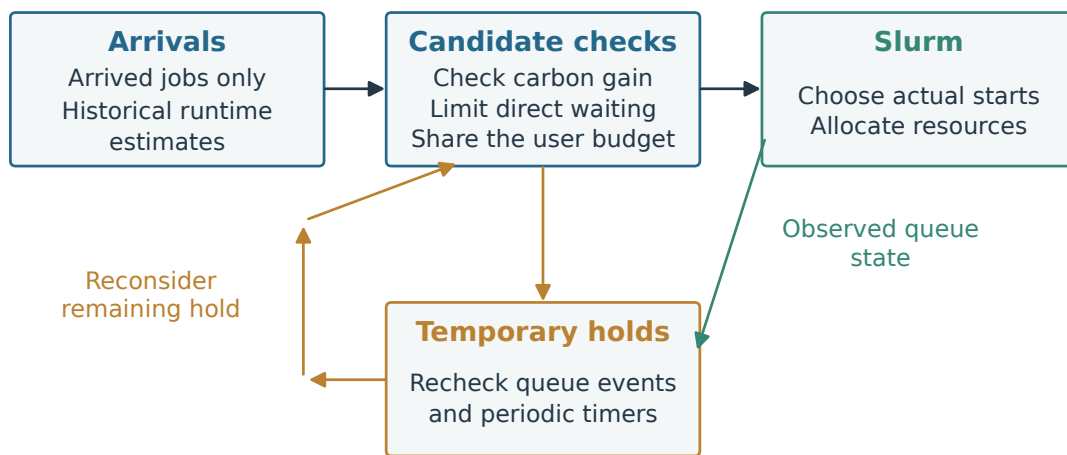
Spending includes waiting already incurred and reservations for currently held jobs. Same-batch job/time alternatives are ordered by estimated benefit per waiting time. When the best option exceeds the remaining user budget, the controller can consider a cheaper feasible option. Cancelling a hold releases its unused reservation, but does not erase delay already experienced.

4.7.2 Connection to actual simulator events

A C bridge in a separate simulator build intercepts job arrivals before submission to Slurm. It passes arrived-job and submitted-job state to a persistent Python controller. The controller can admit a job immediately or retain it outside Slurm for later review. Queue-state changes and timer checks trigger reconsideration: the timer is 300 seconds for the real-record sample and 60 seconds for the functional fixture. Already submitted or running jobs are neither withdrawn nor interrupted.

The controller estimates remaining work from earlier-month runtime predictions and elapsed running time. It does not inspect future arrivals or use the replay's configured completion duration as a decision-time predictor. Its resource-work estimate is deliberately simpler than Slurm: it cannot reproduce priority, fragmentation or backfill reservations exactly. Actual starts and allocations still come from Slurm. The carbon input remains a retrospective NESO series, so this extension removes neither all hindsight nor the need to test decision-time carbon forecasts in a different study.

Virtual-clock advancement is paused while the controller processes an exchange. If the controller fails, the bridge records the failure and submits retained and later-arriving work without further carbon holds. The interface has a 4,096-arrival prototype limit. It is an experimental integration rather than a claim of complete production middleware. Figure 4.4 distinguishes the decision loop from the queue it observes.



Benefit lost or controller fails: release held work.

No withdrawal of submitted jobs; no interruption of running jobs.

Figure 4.4: V2 admission feedback, shown schematically. Only arrived jobs and current Slurm state enter the loop. The controller can cancel remaining delay before submission, but cannot undo queue-position changes already caused by the hold. Runtime estimates use earlier-month history; the carbon series is retrospective.

4.7.3 Bounded checks and their scope

The extension was evaluated using unit/regression tests, a paired three-job functional fixture, a paired sample of 64 trace-derived jobs and an injected controller-startup failure. The total recorded elapsed budget was 1,259.72 seconds, about 21 minutes, including build and debugging attempts. No additional training, full-day replay or five-date campaign was started. The input hashes and results of the original comparison were preserved.

The 64-job sample consists of the first eligible records on 18 November that satisfy the CPU-partition, one-node and 60–3,600-second configured-runtime filters. Both runs begin empty and omit the production carry-in workload. Resources and runtimes are unchanged between the paired runs; 17 of the 64 jobs are flexible under the retained selection. The parameters were fixed before this check: a five-minute candidate grid, a 5% minimum predicted runtime-average intensity improvement and the above job/user budgets. Section 5.6 reports the outcomes separately from the main experiment.

Chapter 5

Evaluation and Results

5.1 Evidence stages

The evaluation developed through several stages, each with a different purpose. Synthetic workloads tested event handling and multi-node configuration. June trace windows supported admission-policy development and exposed the importance of initial queue pressure and complete-runtime carbon exposure. A November comparison examined a broader set of rules, followed by a December low-impact check. The final main experiment placed seven representative strategies on the same five-date comparison. After that campaign closed, a bounded feedback extension tested specific implementation changes.

These stages are not interchangeable test sets. June windows differ in cohort and starting pressure; predictor calibration does not replay a counterfactual queue; and November results informed later rules. Table 5.1 records how each stage is used. This keeps earlier work visible without selecting its best percentages as final evidence.

5.2 Historical comparisons and corrections

5.2.1 Queue fidelity and the November comparison

The 12 November comparison completed twelve scenarios containing 3,900 jobs each: 3,335 evaluation jobs and 565 carry-in jobs. Its source-trace eligible-to-start P95 was 323,424.4 seconds, compared with 53.44 seconds in baseline replay. The queue-fidelity comparison below shows the discrepancy. A complete workload therefore does not establish calibrated production queue timing. Historical priority, fair-share, reservations and administrator state are not fully recovered; simulated waiting is conditional on the reconstruction, not a prediction of actual Stanage service.

Table 5.1: Experimental stages and their role in the dissertation.

Stage	Work completed	Evidential role
Synthetic and June development	Multi-node/partition simulation; trace conversion; initial-state and delay-rule checks.	Development evidence, not the final estimate of savings.
Historical predictors	Earlier-month runtime, queue and load quantiles; later-period calibration diagnostics.	Describes information quality, not measured online carbon-forecast accuracy.
12 November comparison	Twelve scenarios, each with 3,900 jobs.	Historical policy comparison and queue-fidelity diagnostic.
8 December legacy check	Three 5,494-job runs; subsequently audited for initial-state restoration.	Superseded counterfactual evidence because 24 running jobs were not restored at the boundary.
Five-date main campaign	Seven policies per date; three baselines and two Adaptive runs per date; 55 final replays.	Main descriptive comparison, with timing and model limitations reported.
V2 engineering extension	37 code tests, paired three-job and 64-job checks, and a failure check.	Tests implemented mechanisms; not an eighth main-study policy.

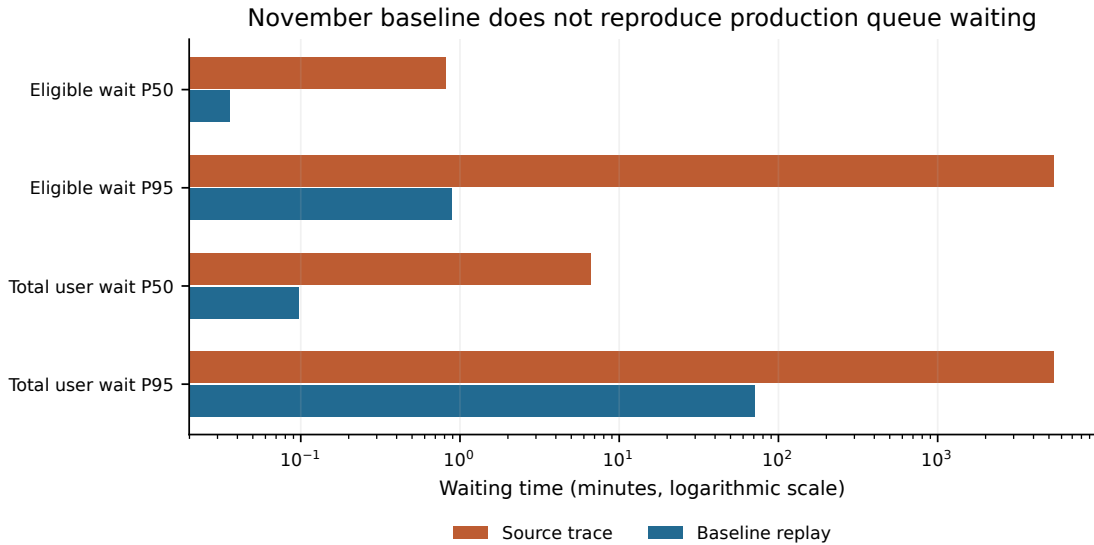


Figure 5.1: Historical preliminary diagnostic. Source-trace and baseline-replay waiting percentiles for the November evaluation cohort. Eligible-to-start P95 was 323,424.4 s in the trace and 53.44 s in replay. The reconstruction does not reproduce production fair-share, reservation and historical-priority state; execution correctness therefore does not establish queue calibration.

The historical comparison remains useful for tracing the policy design. Its paired table and figures distinguish carbon change, tail waiting and cumulative user burden.

Table 5.2: November single-date diagnostic results. Positive carbon reductions mean lower modelled load-related carbon; a positive P95 change means longer total user waiting. All twelve scenarios completed 3,900 jobs.

Policy	Moved	Delay (h)	Capacity (%)	Node (%)	P95 change (min)
Baseline	0	0.00	0.0000	0.0000	0.00
B1 1 h	407	252.45	0.1638	-0.0634	13.24
B1 2 h	420	583.24	0.4283	-0.1220	43.08
B1 4 h	528	1520.23	0.9797	-0.2353	148.89
B2 Q75 / cap50	373	1114.53	0.6566	-0.2330	149.02
Adaptive balanced	191	484.43	0.6819	-0.0239	70.35
08–18 UTC hybrid	445	1402.52	0.8355	-0.2325	168.47
Strict oracle (no move)	0	0.00	-0.0042	-0.0239	0.20
Balanced oracle (no move)	0	0.00	-0.0053	-0.0243	-0.01
History-only	9	28.75	0.1555	-0.2118	0.01
80% consensus	9	28.75	0.1552	-0.2127	-0.01
Safe rolling (no move)	0	0.00	-0.0059	-0.0243	-0.01

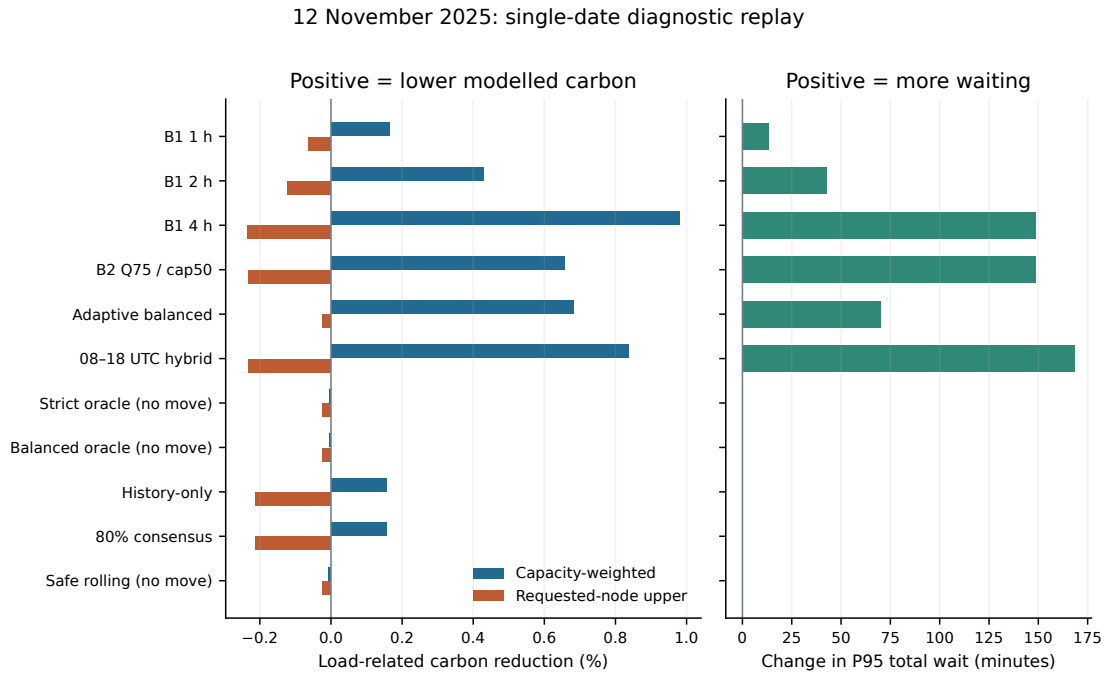


Figure 5.2: Historical preliminary diagnostic. Single-date November replay: carbon-model disagreement and P95 total waiting changes for 3,335 evaluation jobs plus 565 carry-in jobs, with 3,900 completions in every scenario and a common 107.5 h horizon. Positive reduction means (baseline minus policy) divided by baseline. No moving policy improved both proxies. Zero-move oracle and safe differences fall within the observed replay envelope (0.00591% capacity; 0.02432% requested-node) and are not policy effects. November later informed low-impact policy development.

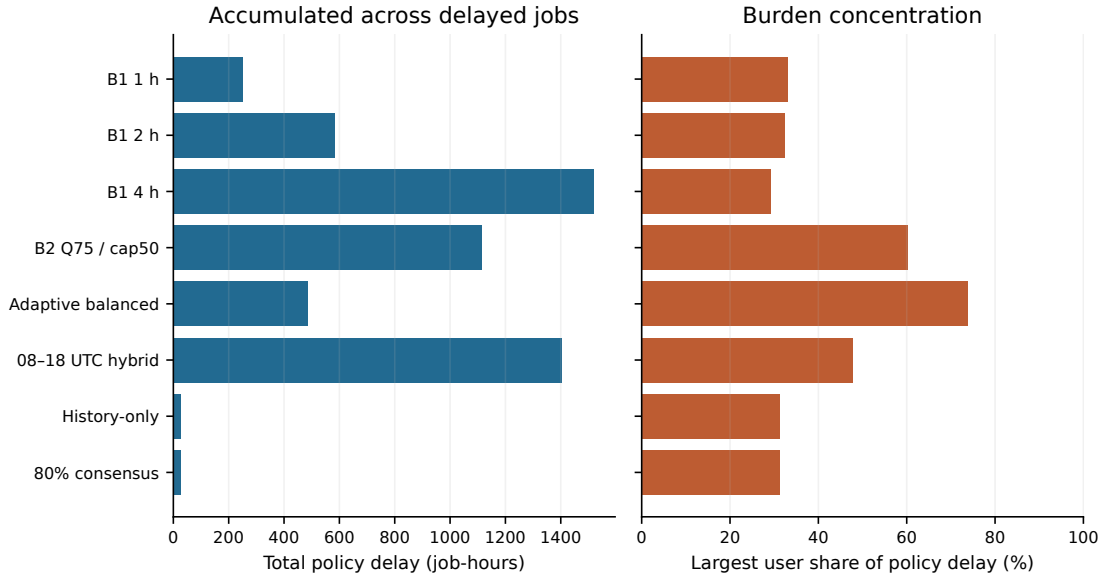


Figure 5.3: Historical preliminary diagnostic. Policy-delay burden in the November single-date replay. Accumulated job-hours sum delays over jobs and are neither experiment duration nor one person’s elapsed waiting time. Adaptive balanced imposed 484.43 job-hours, of which 73.90% fell on one user; satisfying per-job runtime budgets did not establish user-level fairness.

On that date, B1 reduced the primary load-dependent estimate by 0.9797%, added 1,520.23 cumulative job-hours of direct delay and increased P95 total waiting by 148.89 minutes. Adaptive Balanced retained a smaller 0.6819% reduction with 484.43 delay hours and a 70.35-minute P95 increase. Its runtime-related budget therefore offered a less costly trade-off in this historical example. The requested-node changes were instead -0.2353% and -0.0239% , respectively, so neither demonstrates reduction under both representations. The five-date comparison below assesses the same issue on additional matched workloads.

5.2.2 Why the old December percentage is not retained

The earlier 8 December check reported a 0.7179% primary-model increase for Low-impact, alongside lower P95 waiting. A later audit found that 24 jobs that should have been running at the boundary were not restored then; the latest started approximately 12.59 hours later. That mismatch changes the starting resource pressure, so the pair cannot identify the effect of the admission policy. The old files and reported percentage remain in the provenance archive, but their chart is not presented as a valid carbon comparison here. This correction is not evidence that Low-impact succeeded. It is a documented reason to exclude that particular counterfactual claim. The main five-date campaign passed the corresponding initial-state checks and is a separate evidence set.

5.2.3 Historical predictor diagnostics

The earlier runtime/load diagnostics below provide context for the history-based policies. A nominal quantile must be checked against later observations rather than assumed reliable from its name. Short jobs, long GPU work and changes between months can have different prediction errors. These checks informed the historical variants, but do not substitute for observing their queue outcomes after admission.

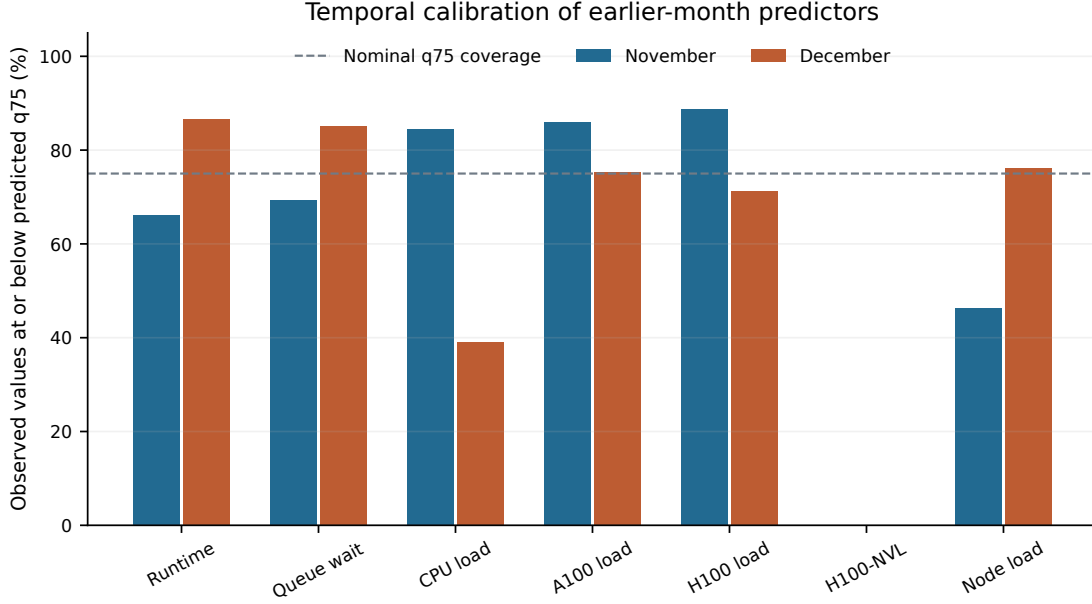


Figure 5.4: Historical preliminary diagnostic. Empirical q75 coverage for history predictors trained on January–October for November and January–November for December. Runtime and queue-wait samples comprise 131,641 and 322,270 valid completed jobs, respectively; load uses 1,440 and 1,488 half-hour intervals. Nominal coverage is 75%. H100-NVL is unsupported by the historical predictor. These are monthly predictor diagnostics, not additional Slurm policy replays or empirical carbon-forecast accuracy.

5.3 Completed five-date comparison

All 55 required final replays completed their expected jobs. The five evaluation populations contain 5,806, 1,629, 5,646, 3,113 and 3,210 jobs, respectively: 19,404 evaluation-job instances in total. Each date also retains its own running, queued and held carry-in work. Different dates preserve their natural counts; within a date, every policy uses the same cohort, runtime, resources, flexibility assignment and initial-state categories. Thus the study compares the policies on the same five cases, rather than testing each on a different favourable subset.

The results use median repeated-baseline carbon and the median of the two Adaptive carbon values. The equal-day normalisations follow Equations 3.8–3.10. The complete daily

tables in Appendix B retain all 35 date–policy combinations, including negative and zero-action observations. Positive reduction values mean lower modelled carbon; negative values mean an increase.

Table 5.3: Five-date descriptive primary results. Positive carbon values are savings; negative values are increases. No policy passes the frozen stability screen.

Policy	Mean (%)	Median (%)	Worst (%)	$\bar{g}$ (g/job)	$\overline{\Delta W}$ (min)	A0
B1 runtime	-0.0784	+0.2205	-2.6427	-0.1250	+161.059	0/5
B2	-0.1371	+0.0199	-1.3488	-0.0825	+18.852	3/5
Adaptive	+0.0649	+0.1696	-0.9680	-0.0240	+19.205	0/5
Day/night hybrid	+0.1161	+0.1758	-0.6582	-0.0038	+31.058	0/5
History-only	+0.1021	+0.2418	-0.3028	+0.0123	+4.969	1/5
Balanced oracle	-0.1882	-0.0190	-0.8069	-0.0709	+8.837	5/5
Low-impact history	-0.2612	+0.0156	-1.3723	-0.1039	+17.479	3/5

All rows contain five complete dates. Mean, median and worst summarize daily primary percentage savings; worst is the minimum. $\bar{g}$ averages $1000K_d/N_d$ equally across dates, where $K_d = C_{b,d} - C_{p,d}$ is cluster saving (kg CO₂). $\overline{\Delta W}$ averages daily paired mean added total wait. A0 counts zero-action dates, whose differences reflect replay variability. Different denominators can make mean percentage and mean normalized saving disagree in sign.

All 35 date-policy combinations | 2025
Primary daily carbon saving (%)

B1 runtime	+1.38	+0.09	+0.55	+0.22	-2.64
B2	+0.57	+0.02 [†]	-0.03	+0.10 [†]	-1.35 [†]
Adaptive	+0.73	+0.00	+0.17	+0.39	-0.97
Day/night hybrid	+0.79	+0.09	+0.18	+0.18	-0.66
History-only	+0.28	+0.01 [†]	+0.24	+0.29	-0.30
Balanced oracle	-0.00 [†]	+0.00 [†]	-0.02 [†]	-0.11 [†]	-0.81 [†]
Low-impact history	+0.02	-0.00 [†]	+0.02	+0.03 [†]	-1.37 [†]
	13 Nov	15 Nov	18 Nov	09 Dec	10 Dec

Teal: saving Amber: increase †: zero action (12 cells)

Figure 5.5: Primary load-related carbon saving for all 35 date-policy pairs. Cells report $100K/C_b$ (%), where $K = C_b - C_p$ is the common-window cluster saving in kg CO₂. Teal indicates saving and amber increase. Signs are retained when values round to zero; the appendix supplies four decimal places. † marks audited zero release actions, whose nonzero differences reflect replay variability rather than policy effects.

5.3.1 Direction, scale and the meaning of an average

B1 reduced the primary load-dependent estimate on four dates, reaching 1.3835% on its best date. On 10 December it instead increased the estimate by 2.6427%. The equal-day mean reduction was consequently -0.0784% . A four-out-of-five success count would miss the importance of this one larger adverse result.

History-only had a positive equal-day mean of 0.1021% in the primary load-dependent model. Dividing each day's shared carbon difference by that day's evaluation-job count, then averaging the five normalised values, gives about 0.0123 g CO₂ per evaluation job. Mean added total wait was 4.97 minutes per evaluation job. These are population summaries. They do not mean each job saved that amount or waited that long, and the carbon figure is not a measured per-job footprint.

The choice of averaging weights matters. Adaptive had a positive mean daily percentage reduction of 0.0649%, but its mean normalised saving was -0.0240 g per evaluation job and its baseline-carbon-weighted reduction was -0.0126% . Dates differ in baseline carbon and job count, so these summaries need not have the same sign. All are retained rather than selecting the favourable one. History-only remained positive under these conventions, with a pooled load-dependent reduction of 0.0813% and a whole-cage reduction of only 0.00335%. The latter illustrates the scale once the fixed component is included; neither percentage represents a measured annual saving.

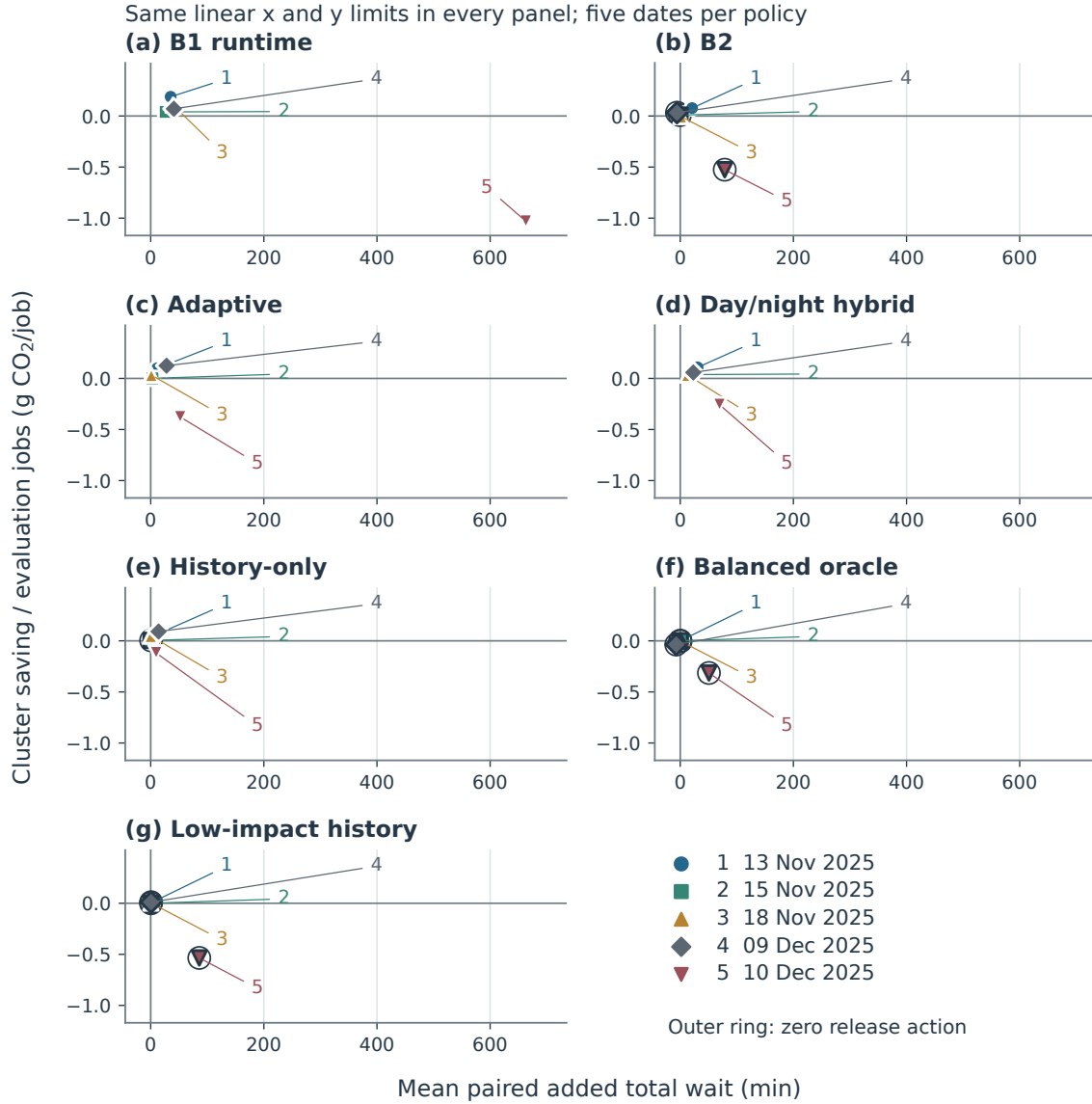


Figure 5.6: Each policy panel retains all five dates, using identical linear axis units and limits. The ordinate is $g = 1000K/N$ (g CO₂ per evaluation job), with cluster saving $K = C_b - C_p$ and evaluation count N ; negative values mean increased carbon. The abscissa is mean paired added total wait in minutes. Numeric labels identify dates. Outer rings mark zero release actions, whose differences reflect replay variability.



Figure 5.7: Complete date-grouped results with common linear limits within each column. Carbon is primary shared-cluster saving $K = C_b - C_p$ (kg CO₂); negative means increase. Wait markers show the mean and P95 of paired total-wait differences in minutes, not differences of separately computed percentiles. AD, HY, HI, OR and LI denote Adaptive, hybrid, history-only, Balanced oracle and low-impact history. † marks zero release actions; their differences reflect replay variability.

5.3.2 Waiting and concentrated user burden

The carbon-wait plots retain each date instead of reducing a strategy to one point. A direct policy hold is only one component of waiting. Changes in queue ordering can affect jobs never selected for delay, and replay variation is also present. Mean waiting differences therefore describe paired observations rather than an isolated causal cost of every selected action.

Across the five dates, B1 deliberately delayed 4,334 evaluation jobs; 3,647 were held longer than their configured runtime. Adaptive delayed 844 jobs, none beyond configured runtime. This verifies Adaptive’s direct-delay bound on the tested jobs. It does not guarantee that scheduler waiting or a user’s cumulative burden is small. History-only delayed 58 jobs, of which 44 exceeded their configured runtime despite meeting predictor-based budgets.

The difference between a population mean and an individual user’s experience is particularly clear for History-only. Its equal-day average direct delay was only 0.523 minutes per evaluation job. On 9 December, however, one pseudonymous user’s 35 jobs accumulated 90.22 job-hours of direct delay and accounted for 86.77% of that day’s policy hold burden. This is a sum over 35 jobs, not a 90-hour hold on one job. The associated overall P95 waiting change was about 21.90 minutes. A small mean does not remove the need for a cumulative user budget.

Direct actions and concentration of positive paired waiting

	(a) Jobs directly held					(b) Largest user's share (%)				
B1 runtime	1026	324	1423	820	741	50.3	47.6	62.1	37.9	48.3
B2	705	0 [†]	260	0 [†]	0 [†]	57.2	31.3 [†]	52.7	26.0 [†]	48.8 [†]
Adaptive	470	73	64	176	61	75.8	61.3	35.3	29.0	53.9
Day/night hybrid	860	252	586	175	81	61.7	39.1	69.3	39.3	39.8
History-only	10	0 [†]	5	41	2	41.9	51.5 [†]	41.8	32.4	34.6
Balanced oracle	0 [†]	0 [†]	0 [†]	0 [†]	0 [†]	62.8 [†]	46.6 [†]	50.6 [†]	21.7 [†]	54.5 [†]
Low-impact history	2	0 [†]	1	0 [†]	0 [†]	65.3	48.5 [†]	52.0	33.7 [†]	52.7 [†]
	11/13	11/15	11/18	12/09	12/10	11/13	11/15	11/18	12/09	12/10

†: zero action; nonzero waiting can still arise from replay variability

Figure 5.8: Direct actions and user-burden concentration for all 35 cases. Panel (a) counts jobs whose releases were intentionally held. Panel (b) reports the largest user's share of positive paired added wait, $100 \max_u B_u / \sum_u B_u$, where $B_u = \sum_{j \in u} \max(\Delta W_j, 0)$; this differs from direct-hold concentration. Panels use separate colour scales. † marks zero actions; apparent waiting burden there reflects replay variability.

Minutes per job in the stated cohort; separate colour scales

(a) Mean direct release hold D : all evaluation jobs

B1 runtime	+36.35	+29.12	+40.31	+41.67	+30.91
B2	+22.02	+0.00 [†]	+1.39	+0.00 [†]	+0.00 [†]
Adaptive	+12.51	+2.00	+1.58	+7.54	+1.15
Day/night hybrid	+31.70	+21.80	+13.62	+8.73	+4.04
History-only	+0.32	+0.00 [†]	+0.16	+2.00	+0.13
Balanced oracle	+0.00 [†]	+0.00 [†]	+0.00 [†]	+0.00 [†]	+0.00 [†]
Low-impact history	+0.02	+0.00 [†]	+0.01	+0.00 [†]	+0.00 [†]
	13 Nov	15 Nov	18 Nov	09 Dec	10 Dec

(b) Mean actual added total wait: all evaluation jobs

B1 runtime	+35.26	+26.45	+39.57	+40.92	+663.10
B2	+21.00	-0.60 [†]	+1.07	-5.81 [†]	+78.60 [†]
Adaptive	+11.62	+3.10	+1.11	+28.05	+52.14
Day/night hybrid	+30.63	+19.36	+12.79	+23.08	+69.43
History-only	+0.40	+0.72 [†]	+0.15	+13.97	+9.61
Balanced oracle	-0.08 [†]	+0.00 [†]	+0.64 [†]	-7.07 [†]	+50.70 [†]
Low-impact history	+0.04	+0.56 [†]	+0.28	+0.66 [†]	+85.85 [†]
	13 Nov	15 Nov	18 Nov	09 Dec	10 Dec

(c) Mean added wait: jobs not directly held

B1 runtime	-1.38	-6.04	-0.23	-6.71	+571.36
B2	-1.22	-0.60 [†]	-0.20	-5.81 [†]	+78.60 [†]
Adaptive	-1.06	-2.42	-0.17	+20.68	+51.75
Day/night hybrid	-1.26	-5.59	-0.31	+22.70	+65.00
History-only	+0.05	+0.72 [†]	+0.05	+10.33	+9.60
Balanced oracle	-0.08 [†]	+0.00 [†]	+0.64 [†]	-7.07 [†]	+50.70 [†]
Low-impact history	+0.02	+0.56 [†]	+0.27	+0.66 [†]	+85.85 [†]
	13 Nov	15 Nov	18 Nov	09 Dec	10 Dec

†: no release action; its apparent spillover is replay variability

Figure 5.9: Direct holds and actual waiting for all 35 cases, in minutes per job. Panels (a–b) average intentional hold D_j and paired added total wait ΔW_j over all evaluation jobs; (c) includes only jobs with $D_j = 0$. Amber means added waiting and teal earlier starts. Different cohort means are not additive components. † marks zero release actions, whose apparent indirect waiting is replay variability, not causal spillover.

5.3.3 No-action cases and repeat variation

Low-impact moved only three evaluation jobs across all five dates, with three cumulative hours of direct delay. The Oracle reference moved none. Its access to a completed baseline schedule did not guarantee a feasible candidate under its gates, and it is not a mathematical optimum. Near-zero intervention is therefore reported as abstention rather than as a successful carbon-control result.

The audit identified twelve no-action policy–date combinations whose executable event, user and configuration files were byte-identical to baseline. Their output differences cannot be attributed to intentional delay. On 10 December, the three baseline primary carbon estimates were 125.1411, 125.0978 and 127.1193 kg, a relative range of 1.6153%. The baseline P95-wait range was approximately 107.40 minutes. That day’s no-action B2, Oracle and Low-impact carbon values all fell inside the observed baseline range. Reporting their negative percentages without the no-action marker would give a misleading mechanism explanation.

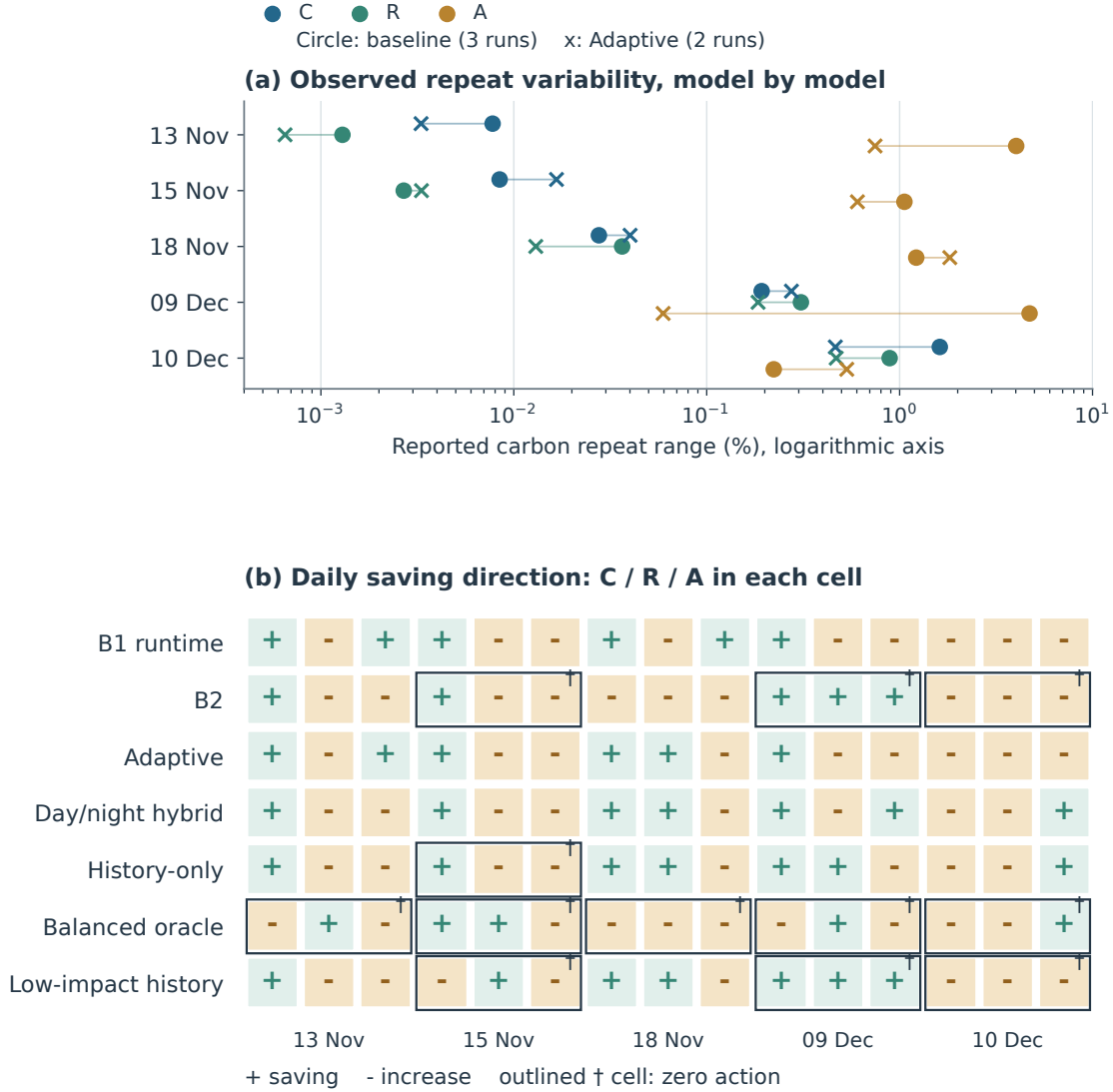


Figure 5.10: (a) Reported within-policy repeat ranges for baseline and Adaptive, with a logarithmic percentage axis. Connecting segments pair descriptive ranges, not confidence intervals. (b) Unrounded saving directions for all 35 cases: C = capacity-weighted, R = node-request upper, A = distinct allocated-node proxies, all load-related. Plus means saving and minus increase; direction alone does not establish robustness. Outlined † cells have zero release actions and reflect replay variability.

5.3.4 Stability under the frozen criteria

No strategy met every frozen carbon-stability condition. B1 exceeded the daily noise and minimum-saving threshold on four dates, but failed on worst-day increase and pooled direction. History-only exceeded that threshold on three dates rather than four, and its worst-day increase was 0.3028%, above the 0.1% tolerance. Its leave-one-date-out mean stayed positive, with a minimum of 0.0562%, which is a useful descriptive indication but does not override the failed conditions.

The seven configurations consequently have mixed or no-action outcomes rather than a single robust ordering. Three occupancy models also disagree frequently: only B2 and Low-impact each had one date positive in all three models; the remaining rules had none. This is sensitivity to allocation representations, not a majority vote among independent meters. The mean, median, spread and worst date describe the cases; with five selected dates and unequal repeats, they are not a basis for an unsupported significance claim.

5.4 Explaining observed carbon increases

An increase requires more than one explanation check. First, was there an actual admission action? Second, were initial state and workload membership matched? Third, how much of the difference is comparable to observed repeat variation? Only after these checks does an interval-level decomposition help explain where the accounting change occurred.

For a policy-minus-baseline difference, write the energy change in interval k as ΔE_k and let $\bar{I}$ be the common window's duration-weighted mean intensity. The following identity separates total-energy and timing contributions:

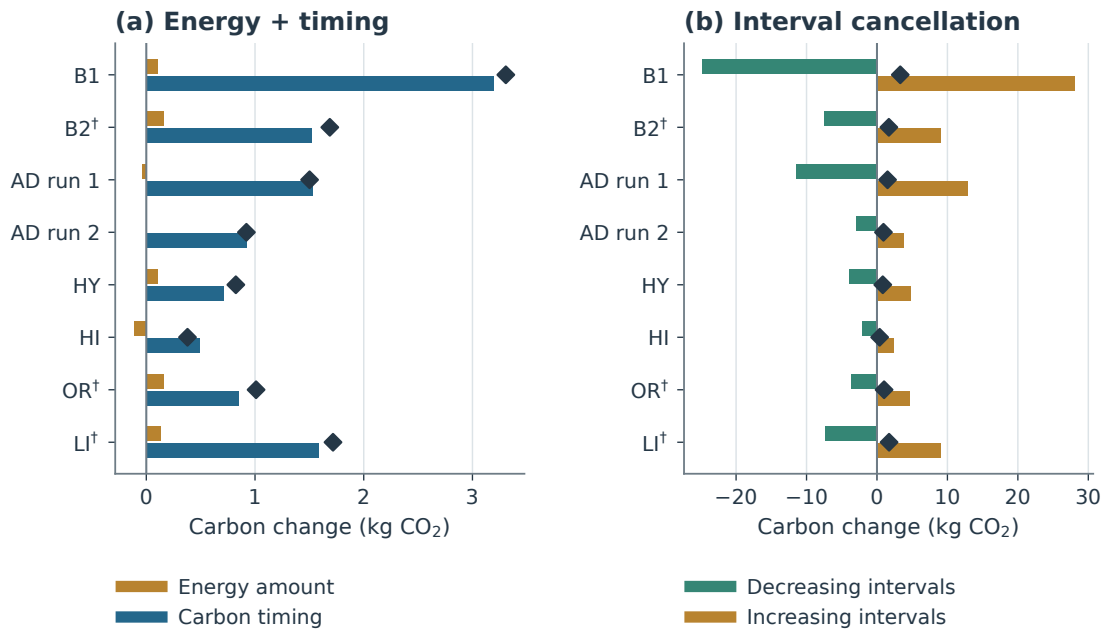
$$\Delta C = \frac{\bar{I} \Delta E}{1000} + \frac{1}{1000} \sum_k \Delta E_k (I_k - \bar{I}), \quad \Delta E = \sum_k \Delta E_k. \quad (5.1)$$

Here ΔE_k and ΔE are modelled load-dependent energy differences in kWh; I_k and $\bar{I}$ have units of g CO₂/kWh; $\Delta C = C_p - C_0$ is in kg CO₂. A positive ΔC means an increase, the opposite sign convention to a reported reduction. The first term is the energy-volume contribution and the second is the time-alignment contribution. Their split depends on the chosen reference intensity, although the net change does not. This is an algebraic decomposition of the scenario outputs, not a causal decomposition of measured power.

Against the selected reference replay on 10 December, B1's 3.3071 kg increase decomposed into approximately 0.1091 kg from energy volume and 3.1979 kg from time alignment. About 96.7% of that net increase therefore lay in the time-alignment term. History-only had a small negative energy-volume contribution but a larger positive time-alignment contribution: consuming slightly less modelled energy did not ensure lower carbon if it occurred in higher-intensity intervals. The decomposition figure uses the saved run pairs, whose reference is stated rather than silently equated with the median-repeat main table.

10 Dec 2025: all seven policies and both Adaptive runs

Positive = increase (opposite sign to the saving figures)



Diamond: exact net change †: zero action Fixed-idle term cancels

Figure 5.11: All eight 10 December policy runs paired with the first baseline, not median headline results. Positive change means increase, $\Delta C = C_p - C_b = -K$. Panel (a) splits $\Delta C = \bar{I}\Delta E/1000 + T$: energy change ΔE (kWh), mean intensity $\bar{I}$ (g CO₂/kWh), and timing residual T (kg CO₂). Panel (b) sums increasing and decreasing intervals. Diamonds give exact net changes; fixed-idle cancels. † denotes zero-action replay variability. This common-window accounting is not causal attribution.

The fixed 140 kW term is not an explanation for these paired increases: its integral is identical over the common horizon and cancels. Occupancy capping, sharing and placement can change the load-dependent energy integral even when job requests and runtimes are unchanged. On 13 November, for example, B1 saved 1.0936 kg in the capacity model while increasing the requested-node estimate by 2.0919 kg. This disagreement limits a hardware-level interpretation.

Queue effects are another plausible mechanism, but cannot be isolated from repeat variation by these runs alone. On 10 December, B1's 741 directly delayed jobs had about 16.15 hours of mean additional waiting against the first baseline; the 2,469 jobs without direct delay also had about 9.52 hours more waiting on average. In the same dated History-only run, only two jobs were held, yet their mean absolute predicted-versus-replayed start error was approximately 35.17 hours. These are case-specific diagnostics, not general prediction-error estimates for the policy.

5.5 Completion, timing and retained failures

Every final replay completed its expected jobs and passed the existing workload and initial-state checks. A later read-only audit nevertheless found at least one logged-runtime error above 60 seconds in 38 of the 55 final replays. Seventeen had no error above that diagnostic line. Completion is therefore reported separately from timing fidelity; the diagnostic was not retroactively used to delete inconvenient dates.

Eight runtime errors exceeded one hour, all in the final extra Adaptive replay on 10 December: seven held carry-in jobs and one evaluation job. Their completion events were processed together after a large gap in the simulation clock. The recorded due times matched logged starts plus configured runtimes within approximately 0.0343 seconds. Crucially, every job had already started before the earliest of these eight events was due. Those particular late completions cannot explain earlier waiting differences. Carbon accounting used configured duration and did not count the entire logged overrun as extra job energy. This narrows the impact of the largest anomaly without proving that all other timing discrepancies are harmless; the clock-gap trigger remains unverified.

The first attempt at that extra Adaptive run had 161 timeouts and was retried under the existing attempt limit. Both attempts' logs and tables remain archived. The reported denominator is 55 final completed replays, not a claim that the entire process had no failures. Similarly, the bridge comparisons supporting parallel execution on 13 November do not prove every later case equally stable or establish that concurrency caused all variation.

5.6 Bounded feedback-extension results

5.6.1 Code behaviour and the functional fixture

All 37 final V2 unit/regression tests passed. They covered feasible alternative candidates, immediate admission, user-budget allocation, spent-delay accounting, repeated submission

prevention, unseen-arrival isolation, carbon coverage, the total experiment deadline and refusal to restart a closed experiment. These checks validate the specified behaviours, not the real-world effectiveness of the policy.

In the paired three-job fixture, one job received a hold to 300 seconds, another job from the same user was rejected by the shared budget, and a larger job arrived at 60 seconds. The controller cancelled the first job's remaining hold near 60.10 seconds. Both runs completed all three jobs, but cancelling the hold did not restore its queue position: that job's total wait rose from 3.24 to 1,322.36 seconds. It was submitted behind the larger job and then waited for resources. Figure 5.12 makes this difference visible.

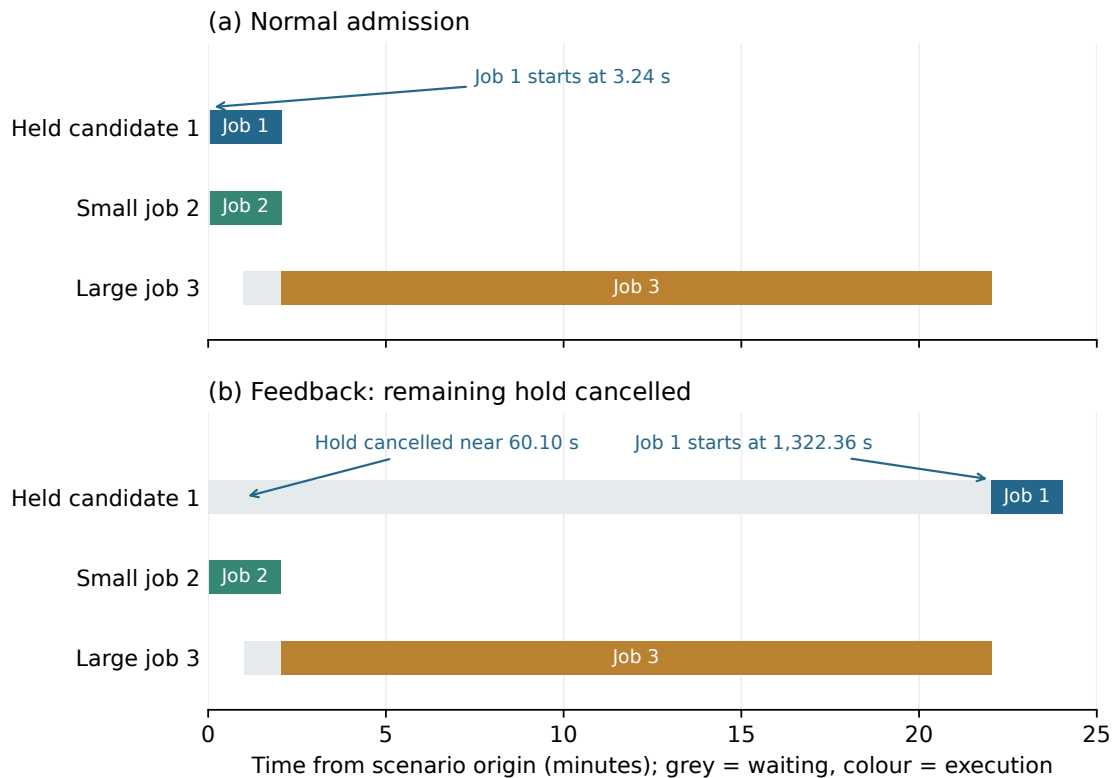


Figure 5.12: Paired three-job functional fixture, not a Stanage performance result. Bars use accounting start and configured runtime. Job 1 was initially held for release at 300 seconds; a larger job arrived at 60 seconds and the controller cancelled the remaining hold near 60.10 seconds. Job 1 nevertheless entered behind that large job and waited 1,322.36 seconds in total, compared with 3.24 seconds under normal admission. Earlier release did not restore the earlier queue position.

This fixture's capacity carbon proxy fell by about 8.45%, while mean waiting increased by approximately 439 seconds. The proxy was used to check the calculation path in a designed scenario. It is not a Stanage saving and is excluded from the seven-policy result table. The

substantive finding is narrower: cancellation limits remaining direct hold, but cannot guarantee recovery from a queue-order change already caused.

5.6.2 The paired 64-job trace sample

The sample used the same 64 job records and empty initial state in both runs. Both completed all jobs and neither deliberately held any. Of the feedback jobs, 47 were admitted under protection rules and 17 flexible jobs had no feasible hold. Twenty candidate times failed the 5% improvement gate; some short jobs also had budgets too small to reach the next five-minute candidate. Abstention here is an observed response to the fixed rules, not a reason to lower the threshold after seeing the result.

Table 5.4: V2 paired 64-job engineering check. Waiting and runtime errors are in seconds.

Metric	Normal admission	Feedback
Jobs completed	64/64	64/64
Jobs deliberately held	0	0
Mean total wait (s)	1,341.38	1,342.76
P95 total wait (s)	2,362.14	2,411.63
Maximum absolute runtime error (s)	54.66	66.08
Jobs with runtime error above 60 s	0	1

The timing-alignment carbon proxy changed by -0.01321% when expressed as a reduction, while mean wait increased by 1.39 seconds. As no job was actively held, these are not carbon or waiting effects of intentional delay. A single pair cannot separate controller processing from replay variation. The proxy integrates capacity fraction, duration and carbon intensity; it omits the main study’s 140/195 kW conversion and full carry-in queue. It is not a physical per-job footprint or a replacement whole-cage estimate.

One feedback job ran for 279.08 logged seconds against a configured 213 seconds, exceeding the unchanged 60-second timing diagnostic. The warning remains even though completion passed. A separate injected controller-configuration failure triggered the bridge’s fail-open path and all three fixture jobs completed. This covers that startup failure, not every failure mode. Successful simulation attempts still showed the existing exit code 139 after completion, so process shutdown was not reported as clean.

An earlier functional attempt exposed an arrival-grid budget error, which was corrected before the accepted fixture. The first real-record attempt could not resolve simulated nodes under an added network-isolation setting; it was stopped and excluded. Restoring the project’s standard container network allowed the same input to run. These attempts remain recorded, and the combined work finished within about 21 minutes of the single three-hour budget. The extension is complete at this bounded scope; no new full-day experiments are required to interpret these observations.

Chapter 6

Discussion

The completed five-date comparison gives mixed results for carbon-aware admission. Some policies reduce estimated carbon on several dates, but none meets every frozen stability condition. The discussion therefore considers the size of the observed changes alongside waiting, user burden and replay variation. The later V2 tests answer a separate engineering question about candidate selection and feedback from the simulated queue.

6.1 Answers to the research questions

6.1.1 RQ1: Changes in modelled carbon

The policies did not retain the same ordering across dates. Runtime-aware B1 reduced primary load-dependent carbon on four dates, with a best result of 1.3835%. Its 2.6427% increase on 10 December was large enough to leave the equal-day mean reduction at -0.0784% . The four positive dates alone would give a misleading account of this policy. B2 also had a negative equal-day mean, while Adaptive Balanced and the day/night hybrid had small positive mean percentage reductions.

History-only produced a positive primary-model mean under several aggregation methods. Its equal-day mean reduction was 0.1021%. Dividing each date’s shared-queue saving by its evaluation-job count and then weighting the dates equally gave 0.0123 g CO₂ per evaluation job. This is a normalised cluster difference, rather than a measured footprint for an individual job. History-only is worth retaining as a candidate for discussion, although its small average does not establish a consistent benefit: it exceeded the material-saving and repeat-noise threshold on three dates rather than four, and its worst date showed a 0.3028% increase.

The comparison answers RQ1 for the tested configurations and workloads. It shows that a lower-intensity admission plan can produce either sign of estimated carbon change after Slurm has scheduled the shared queue. This is compatible with the distinction between ideal temporal opportunity and practically constrained shifting discussed by Sukprasert and colleagues [25]. The local question is whether the resulting execution intervals improve carbon exposure, rather than whether the selected release timestamp has lower intensity.

6.1.2 RQ2: Waiting and user burden

The central service question is how much waiting accompanies a carbon change and who bears that waiting. Adaptive Balanced deliberately delayed 844 evaluation jobs, with none delayed longer than its configured runtime. B1 delayed 4,334 jobs, including 3,647 with direct delay greater than runtime. This verifies the effect of Adaptive’s runtime-proportional bound on the chosen admission actions. Across dates, its mean observed additional total wait was 19.20 minutes per evaluation job, compared with 161.06 minutes for B1. The smaller direct-delay budget did not by itself make all of Adaptive’s carbon or service outcomes favourable.

History-only had a much smaller equal-day mean direct delay of 0.523 minutes per evaluation job, but its mean observed additional total wait was 4.97 minutes. The user-level results give a different view of this average. On 9 December, one user’s 35 jobs accumulated 90.22 hours of direct delay and accounted for 86.77% of that date’s total. These are cumulative job-hours, not a single job waiting for 90 hours. A small population mean can therefore coexist with a substantial burden on one user.

The user-burden distribution also has a narrower interpretation than a user carbon account. Wassermann and colleagues show that carbon attribution requires choices about measurement and idle-resource responsibility [27]. Here, the observable distribution concerns imposed waiting. It supports statements about who experiences a service cost, while the carbon outcome is assessed for the shared cage scenario.

These results do not imply that every carbon reduction necessarily requires worse service for every user. Direct policy delay is non-negative, but changes in total waiting can have either sign as jobs compete for resources. Replay variation also contributes to the observed differences. The paired results consequently describe changes in the simulated queue without assigning every change to a deliberate policy action. Positive-only additional waiting exposes later starts that earlier starts elsewhere could hide. A favourable population P95 likewise does not mean that each affected job benefited.

6.1.3 RQ3: Information and delay safeguards

Observed runtime helps identify whether moving a non-preemptive job improves its complete execution exposure. A completed baseline schedule additionally reveals future queue and load information. These advantages explain why the given-runtime policies and Oracle reference are useful for diagnosis, but their information conditions differ from those of the historical predictors.

Earlier-month quantiles remove some of that hindsight from decision runtime and load. Their performance depends on calibration, support and temporal change. History-only delayed 58 evaluation jobs, of which 44 received direct delays longer than their configured execution runtime. A policy can obey a budget based on a prediction while imposing a large delay relative to the eventual job duration. The representative Oracle and History-only rules also have different thresholds and weights, so their comparison does not isolate the effect of prediction error alone.

The Low-impact rule delayed only three evaluation jobs across the five dates, while the Oracle reference made no delay action. These outcomes describe the behaviour of the implemented selection rules. They do not show that the workloads contain no useful shifting opportunity, and the Oracle rule is not a mathematical upper bound. The V2 candidate-selection correction adds another qualification: ranking before testing all constraints can miss a feasible alternative. Its regression evidence concerns code behaviour and does not supply revised results for the frozen policies.

All methods use a historical carbon series without archived issue-time forecasts. The history-only label therefore describes the runtime and load information boundary, not a complete online forecast evaluation. Forecast sensitivity in temporal-shifting research provides a reason to examine uncertainty, but an assumed noise process has a different evidential status from a measured forecast history [29]. Runtime, load and carbon information need separate treatment because their provenance and timing restrictions differ.

6.1.4 RQ4: Consistency across dates and assumptions

The mean, median, worst date and observed repeat range answer different questions. An equal-day mean gives each selected case the same weight, whereas the median describes the middle result and is less affected by one extreme date. The worst date records an adverse outcome that neither summary should hide. B1's median reduction was positive at 0.2205%, even though its mean was negative. Reporting both explains the distribution more clearly than choosing whichever statistic favours the policy. The standard deviation and interquartile range describe cross-date spread; repeated-run ranges describe variation within a date.

Aggregation also changes the question being answered. Adaptive's mean daily percentage reduction was 0.0649%, but its equal-day mean saving per evaluation job was -0.0240 g CO₂, and its pooled baseline-carbon-weighted reduction was -0.0126% . Daily baseline emissions and job counts differ, so these summaries use different weights and need not have the same sign. The normalised carbon and waiting means use all evaluation jobs, including those without deliberate delay. A mean of daily P95 values is not a pooled job-level P95, and a difference between two population P95 values is not the P95 of paired job-level differences.

No policy met the full frozen carbon screen. It required five valid dates, material positive changes on at least four, a worst-day increase no greater than 0.1%, a positive pooled reduction and a positive mean after leaving out any one date. History-only's lowest leave-one-date-out mean remained positive at 0.0562%, but it failed the required number of material-positive dates and the worst-day limit. The service screen was separate. These are descriptive project criteria, not a statistical significance test, and the negative dates remain in every five-date summary.

Identical-input runs show why the repeat range matters. On 10 December, the three primary-model baseline estimates had a relative range of 1.6153%. Across the campaign, 12 no-action policy-date combinations had executable inputs identical to their baseline. Their carbon and waiting differences cannot be effects of intentional delay. In particular, the Oracle's observed mean additional wait is not a policy-imposed waiting budget. Three baseline runs

and two Adaptive runs per date reveal some variation, but the single replay of each other policy does not establish its full run-to-run distribution.

6.2 What the carbon model supports

The framework compares counterfactual schedules under a shared power scenario. It does not measure the electricity consumed by individual Stanage jobs. Resource allocation is observable in the trace, while application activity and electrical demand are not. The mapping from normalised resource occupancy to the 55 kW load-dependent range is consequently a modelling assumption.

On the common model-completion horizon, the fixed 140 kW term contributes the same integral to each replay and cancels from the paired absolute difference. It remains in the whole-cage percentage denominator. Extra waiting therefore does not automatically add a different fixed idle-energy charge to the policy. For History-only, the sum of paired savings was approximately 0.3922 kg CO₂, equivalent to a pooled reduction of 0.0813% in load-dependent carbon but only 0.00335% in whole-cage carbon. The execution windows overlap, so this sum is not a continuous five-day emissions account or a basis for annualising savings.

The reporting distinction is consistent with the separate dynamic and infrastructure components in the BLOOM footprint study [14]. It does not imply that the relative sizes of those components are the same on Stanage. Here the fixed term comes from the stated cage scenario, so the load-dependent percentage is not a substitute for the whole-cage percentage.

Disagreement between capacity, node-request and distinct-node views is model sensitivity. History-only's positive primary-model mean was accompanied by negative mean reductions under both alternative occupancy models. Selecting the favourable proxy would hide this dependence. A calibrated aggregate power series could help establish which occupancy representation tracks electrical behaviour more closely. The current alternatives show how the conclusion changes with the allocation model; they are not independent electrical measurements.

Trace-based estimation and HPC power-modelling research both make the adequacy of the resource-to-power mapping a separate issue from scheduling correctness [18, 28]. This is why the three occupancy views are retained together. Choosing one after observing which gives the largest reduction would change the meaning of the comparison.

The regional carbon signal is an accounting proxy for electricity supply. It does not measure a particular job's causal effect on generation. The literature distinguishes operational accounting from broader lifecycle and consequential questions [8, 11, 25]. The dissertation reports generation CO₂ within the stated cage-power boundary. Legacy C02e field names do not extend that boundary to a lifecycle inventory, and no additional PUE multiplier is applied.

In the terminology used by Dodge and colleagues, this is an attributional comparison, while the consequential effect on generation would require marginal-emissions evidence [7].

Agreement among occupancy models does not remove that distinction. It checks an allocation assumption inside the experiment, not the causal response of the electricity system.

6.3 Replay validity and generalisation

Identical workloads, shared accounting horizons and complete initial-state checks support comparison within the model. Completion and timing fidelity nevertheless require separate judgements. All 55 final replays completed their jobs, but 38 had at least one logged runtime error greater than 60 seconds. Section 5.5 reports eight errors above one hour in the final Adaptive repeat on 10 December: seven held carry-in jobs and one evaluation job. All jobs had already started before the earliest of these completion events was due. Their late processing therefore cannot explain earlier waiting differences, and the carbon model did not integrate their full logged overrun as extra execution energy. Other timing differences remain part of the uncertainty, and the trigger of the large clock gap is unresolved.

The final Adaptive repeat also required a retry after 161 jobs timed out in its first attempt. The failed attempt is retained, while the successful attempt supplies the final result. Thus 55 completed replays is the final comparison denominator, not a statement that every attempt succeeded. The older 8 December pairing has a different validity problem: 24 initially running jobs were restored late in the policy run. Its recorded carbon increase and P95 decrease remain historical audit observations, but neither supports a causal explanation about that policy. This invalid pairing is separate from the five-date results.

The use of repeated schedules is supported by the variability discussed in the later Slurm simulator work [22]. It addresses one source of internal uncertainty. It cannot compensate for a missing production reservation, incorrect hardware reconstruction or an uncalibrated power model; physical-system validation is a separate evidential step [9].

The five dates form a designed case study of one cluster, not independent random samples of the year. Carry-in work and execution tails can overlap. The assumed flexibility assignment and the difference between reconstructed and production queue state also affect the interpretation. The study therefore describes consistency across these cases without estimating an annual population effect.

The supported completed-work cohort is narrower than the production workload. Failed and cancelled jobs may consume resources without appearing as equivalent replayable jobs here. Runtime caps and carry-in reconstruction also change how source records enter the model. Resource requests and configured runtimes are held fixed between paired policies after this conversion. A complete replay denominator establishes completion of the selected model workload, rather than reproduction of every production job.

6.4 What the feedback extension adds

The bounded V2 work followed the frozen campaign as a separate engineering extension. It implemented feasibility checks before candidate ranking and a controller connected to the

actual Slurm simulator event loop. Figure 4.4 shows how arrived jobs and current queue states enter that loop. Held jobs can be released when their expected benefit disappears, while spent waiting remains charged to the user budget. This differs from generating a static release plan before replay. The original seven-policy outcomes remain unchanged.

The three-job functional test gives a concrete limit to early release. A job initially selected for submission at 300 seconds was released at about 60.10 seconds of simulated time, following a queue-state change. Its total wait nevertheless increased from about 3.24 seconds in the paired baseline to 1,322.36 seconds, because a larger job had entered the queue ahead of it. Figure 5.12 separates the direct hold from the subsequent queue wait. Cancelling the remaining delay did not restore the earlier queue position. This fixture demonstrates an interaction between admission and queue order; its carbon proxy change is not a Stanage performance result.

Section 5.6 also reports the paired 64-job check drawn from 18 November. Both runs started empty and used the same selected single-node CPU jobs, without the full carry-in workload. The controller made no deliberate holds. The mean wait difference of about 1.39 seconds and the proxy reduction of -0.01321% therefore describe differences in a no-action pair, not an effect of shifting jobs. The single pair does not separate controller-processing effects from replay variation. One feedback-run job exceeded the 60-second timing diagnostic. This check supports the operation of the interface and its rejection rules, while repeatability and full-day carbon performance were outside its scope.

The saved test log records 37 passing tests, including budget accounting and feasible alternative selection. These tests and the functional fixture support specific claims about implementation behaviour. The budgets constrain direct admission waiting, whereas actual queue waiting still depends on Slurm and later arrivals. The V2 carbon proxy also lacks the power scaling of the main 140/195 kW model, so its values remain separate from the five-date carbon tables.

6.5 Practical implications and future work

The project supports an inspectable comparison of admission rules and makes their service costs visible. The results favour evaluating each proposed action over its execution interval, checking cumulative user burden and allowing a rule to abstain when its constraints are not met. They also show why direct-delay budgets need to be assessed against realised queue outcomes. These are findings about the design and evaluation of the tested policies, rather than a claim that one configuration is best for every workload.

The frozen comparison and the subsequent bounded V2 checks are complete, and the experimental phase is closed. Future studies could align aggregate power measurements with the same cage boundary, retain carbon forecasts at issue time, and obtain user-approved definitions of flexible work. Further investigation of replay variation and queue feedback would also build on the retained evidence. Such extensions are separate research questions, not unfinished experiments required to complete this dissertation.

Chapter 7

Conclusion

This dissertation evaluated carbon-aware job admission using Stanage accounting records and trace-driven Slurm simulation. It developed a workflow for auditing the data, reconstructing initial workload state, applying admission policies and comparing carbon with service outcomes. Slurm supplied the scheduling and resource allocation functions. The contribution is the implementation and case-study evaluation around that simulator, rather than a new scheduling engine or a measured account of Stanage electricity use.

The data audit examined 4,114,698 source records and identified a deduplicated cohort of 4,098,801 jobs submitted during 2025. It established which timing and resource fields could support replay and found no usable non-zero energy measurements in the supplied fields. A stated cage-power scenario therefore provided the basis for carbon estimation. The main comparison covered seven policies on five UTC dates, with three baseline runs and two Adaptive Balanced runs per date, giving 55 final completed replays. The analysis retained mixed and no-action outcomes alongside positive results.

The answer to RQ1 is that carbon changes depended on both policy and date. B1 reduced the primary estimate on four dates but had a negative equal-day mean after its worst result was included. History-only produced a small positive mean of 0.1021% for load-dependent carbon, or 0.0123 g CO₂ per evaluation job under equal-day normalisation. None of the seven configurations met every frozen stability condition. The evidence supports discussion of small modelled gains in particular cases, without establishing a consistently best policy.

RQ2 showed that direct delay and total waiting need separate assessment. Adaptive’s runtime-proportional bound constrained the direct delay of all 844 jobs it moved. That bound did not guarantee a small queue response or low cumulative burden. History-only’s mean observed additional total wait was 4.97 minutes per evaluation job, yet one user accumulated 90.22 hours of direct delay across 35 jobs on 9 December. The cost of a policy cannot be represented fully by an average over all jobs. Equally, the results do not establish that every reduction in carbon must make every user’s service worse.

For RQ3, earlier-month predictors reduced the use of observed test-job runtime and future load information, but prediction-based budgets still differed from bounds based on execution runtime. The conservative Low-impact rule moved only three jobs, and the Oracle reference

made no delay action. Their behaviour reflects the tested information inputs and selection rules, not the absence of all shifting opportunities. All policies shared retrospective carbon data, and their different safeguards prevent an interpretation based on information accuracy alone.

RQ4 identified dependence on the occupancy model and on replay variation. Mean, median and worst-date results exposed different aspects of the five cases, while repeated runs showed differences even without policy actions. The fixed idle-power term cancelled from absolute differences over common model-completion horizons but reduced the scale of whole-cage percentage savings. Timing audits also separated job completion from event accuracy. These checks define the evidence supporting the conclusions and prevent a small favourable percentage from carrying more weight than the experiment allows.

The separate V2 extension added feasible-first candidate selection, explicit decision records and event-driven admission feedback with cumulative user budgets. Its functional test showed that releasing a held job early may still leave it behind work that has already entered the Slurm queue. The selected 64-job real-record pair made no deliberate holds, so it supplied engineering evidence about the interface and rejection behaviour rather than a new carbon-performance result. V2 neither replaces the frozen comparison nor adds an eighth policy to its result table.

The study's main outcome is an auditable evaluation of how carbon estimates, admission decisions and shared-queue waiting interact in this Stanage case. It identifies useful effects of direct-delay safeguards as well as the limits of judging policies by a single carbon or waiting statistic. The experimental phase is closed. Calibrated power measurements, issue-time carbon forecasts and further study of replay variation and feedback remain possible extensions for later research, rather than unfinished requirements of this dissertation.

Bibliography

- [1] ANTHONY, L. F. W., KANDING, B., AND SELVAN, R. Carbontracker: Tracking and predicting the carbon footprint of training deep learning models. arXiv:2007.03051, 2020. Presented at the ICML Workshop on Challenges in Deploying and Monitoring Machine Learning Systems.
- [2] ANTICI, F., BARTOLINI, A., DOMKE, J., KIZILTAN, Z., AND YAMAMOTO, K. F-DATA: A Fugaku workload dataset for job-centric predictive modelling in HPC systems. *Scientific Data* 12 (2025), 1321.
- [3] BARTHOLOMEW, S. L., COLLING, L., DASGUPTA, A., GREENBERG, A. J., LANNE-LONGUE, L., LESTANG, T., MARTINEZ-ORTIZ, C., SAUZÉ, C., WALKER, A. M., AND WARD, A. S. CATS: The climate aware task scheduler. *Journal of Open Source Software* 10, 111 (2025), 8251.
- [4] CATS CONTRIBUTORS. CATS 1.1.0 documentation: Use with schedulers. Online software documentation, <https://greenscheduler.github.io/cats/use-with-schedulers.html>. Accessed 8 September 2026.
- [5] CHADHA, M., JOHN, J., AND GERNDT, M. Extending SLURM for dynamic resource-aware adaptive batch scheduling. In *2020 IEEE International Conference on High Performance Computing, Data, and Analytics (HiPC)* (2020), IEEE.
- [6] CZARNUL, P., PROFICZ, J., AND KRZYWANIAK, A. Energy-aware high-performance computing: Survey of state-of-the-art tools, techniques, and environments. *Scientific Programming* 2019 (2019), 8348791.
- [7] DODGE, J., PREWITT, T., TACHET DES COMBES, R., ODMARK, E., SCHWARTZ, R., STRUBELL, E., LUCCIONI, A. S., SMITH, N. A., DECARIO, N., AND BUCHANAN, W. Measuring the carbon intensity of AI in cloud instances. In *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency* (2022), ACM.
- [8] GUPTA, U., ELGAMAL, M., HILLS, G., WEI, G.-Y., LEE, H.-H. S., BROOKS, D., AND WU, C.-J. ACT: Designing sustainable computer systems with an architectural carbon modeling tool. In *Proceedings of the 49th Annual International Symposium on Computer Architecture* (2022), ACM, pp. 784–799.

- [9] JOKANOVIC, A., D’AMICO, M., AND CORBALAN, J. Evaluating SLURM simulator with real-machine SLURM and vice versa. In *2018 IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)* (2018), IEEE, pp. 72–82.
- [10] KAMIYA, G., AND COROAMĂ, V. C. Data centre energy use: Critical review of models and results. Tech. rep., EDNA Platform, Technology Collaboration Programme on Energy Efficient End-Use Equipment (4E TCP), Mar. 2025. <https://www.iea-4e.org/edna/publications/data-centre-energy-use-critical-review-of-models-and-results/>.
- [11] LANNELONGUE, L., GREALEY, J., AND INOUE, M. Green algorithms: Quantifying the carbon footprint of computation. *Advanced Science* 8 (2021), 2100707.
- [12] LECHOWICZ, A., SHENOY, R., BASHIR, N., HAJIESMAILI, M., WIERMAN, A., AND DELIMITROU, C. Carbon- and precedence-aware scheduling for data processing clusters. arXiv:2502.09717, 2025.
- [13] LI, B., BASU ROY, R., WANG, D., SAMSI, S., GADEPALLY, V., AND TIWARI, D. Toward sustainable HPC: Carbon footprint estimation and environmental implications of HPC systems. arXiv:2306.13177v4, 2023. Version 4, 19 November 2023; local version used for the methodological comparison.
- [14] LUCCIONI, A. S., VIGUIER, S., AND LIGOZAT, A.-L. Estimating the carbon footprint of BLOOM, a 176b parameter language model. *Journal of Machine Learning Research* 24, 253 (2023), 1–15.
- [15] LUO, M., ZHUANG, S., VENGADESAN, S., BHARDWAJ, R., CHANG, J., FRIEDMAN, E., SHENKER, S., AND STOICA, I. Starburst: A cost-aware scheduler for hybrid cloud. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)* (2024), USENIX Association, pp. 37–57.
- [16] NATIONAL ENERGY SYSTEM OPERATOR. Carbon intensity API: About and methodology. Official service documentation, <https://carbonintensity.org.uk/>. Accessed 8 September 2026. Reports electricity-generation CO2 intensity, not a complete life-cycle greenhouse-gas inventory.
- [17] NATIONAL ENERGY SYSTEM OPERATOR. Carbon intensity API for great britain: Regional endpoints and region identifiers. Official API documentation, <https://api.carbonintensity.org.uk/>. Accessed 8 September 2026; region 5 is South Yorkshire.
- [18] O’BRIEN, K., PIETRI, I., REDDY, R., LASTOVETSKY, A., AND SAKELLARIOU, R. A survey of power and energy predictive models in HPC systems and applications. *ACM Computing Surveys* 50, 3 (2017), 37:1–37:38.

- [19] SCHEDMD. Slurm workload manager: sbatch. Official command reference, <https://slurm.schedmd.com/sbatch.html>. Accessed 8 September 2026; the `-begin` option defers allocation eligibility and does not guarantee an execution start time.
- [20] SCHEDMD. Slurm workload manager: Scheduling configuration guide. Official software documentation, https://slurm.schedmd.com/sched_config.html. Accessed 8 September 2026; describes scheduling semantics, not Stanage’s historical site configuration.
- [21] SIMAKOV, N. A., DELEON, R. L., INNUS, M. D., JONES, M. D., WHITE, J. P., GALLO, S. M., PATRA, A. K., AND FURLANI, T. R. Slurm simulator: Improving Slurm scheduler performance on large HPC systems by utilization of multiple controllers and node sharing. In *Practice and Experience in Advanced Research Computing (PEARC ’18)* (2018), ACM.
- [22] SIMAKOV, N. A., DELEON, R. L., LIN, Y., HOFFMANN, P. S., AND MATHIAS, W. R. Developing accurate Slurm simulator. In *Practice and Experience in Advanced Research Computing (PEARC ’22)* (2022), ACM.
- [23] SIMAKOV, N. A., INNUS, M. D., JONES, M. D., DELEON, R. L., WHITE, J. P., GALLO, S. M., PATRA, A. K., AND FURLANI, T. R. A Slurm simulator: Implementation and parametric analysis. In *High Performance Computing Systems. Performance Modeling, Benchmarking, and Simulation*, vol. 10724 of *Lecture Notes in Computer Science*. Springer, 2018, pp. 197–217. PMBS 2017 proceedings; 2018 imprint used as printed in the local chapter.
- [24] SOUTER, N. E., LANNELONGUE, L., SAMUEL, G., RACEY, C., COLLING, L. J., BHAGWAT, N., SELVAN, R., AND RAE, C. L. Ten recommendations for reducing the carbon footprint of research computing in human neuroimaging. *Imaging Neuroscience* (2023).
- [25] SUKPRASERT, T., SOUZA, A., BASHIR, N., IRWIN, D., AND SHENOY, P. On the limitations of carbon-aware temporal and spatial workload shifting in the cloud. In *Proceedings of the Nineteenth European Conference on Computer Systems* (2024), ACM, pp. 924–941.
- [26] THE UNIVERSITY OF SHEFFIELD. Stanage specifications. Sheffield HPC Documentation, https://docs.hpc.shef.ac.uk/en/latest/stanage/cluster_specs.html. Accessed 8 September 2026. Current specifications; not a dated inventory of the 2025 trace.
- [27] WASSERMANN, C., BIELERT, M., VANBERG, G., HACKENBERG, D., TERBOVEN, C., AND MÜLLER, M. S. Calculating user-centric carbon footprints for HPC. In *2024 IEEE International Conference on Cluster Computing Workshops* (2024), IEEE, pp. 26–35.
- [28] WEST, K., REID, M., ELKHATIB, Y., AND THAMSEN, L. Ichnos: A carbon footprint estimator for scientific workflows. arXiv:2411.12456v3, 2025. Version 3, 6 August 2025; first submitted in 2024 and associated with LOCO 2024.

- [29] WIESNER, P., BEHNKE, I., SCHEINERT, D., GONTARSKA, K., AND THAMSEN, L. Let's wait awhile: How temporal workload shifting can reduce carbon emissions in the cloud. In *Proceedings of the 22nd International Middleware Conference* (2021), ACM, pp. 260–272.
- [30] XU, K., SUN, D., TIAN, H., ZHANG, J., AND CHEN, K. GREEN: Carbon-efficient resource scheduling for machine learning clusters. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)* (2025), USENIX Association, pp. 999–1014.
- [31] YANG, Z., MENG, L., CHUNG, J.-W., AND CHOWDHURY, M. Chasing low-carbon electricity for practical and sustainable DNN training. Workshop paper, Tackling Climate Change with Machine Learning at ICLR 2023. arXiv:2303.02508v2, 2023. <https://arxiv.org/abs/2303.02508v2>.
- [32] YOU, J., CHUNG, J.-W., AND CHOWDHURY, M. Zeus: Understanding and optimizing GPU energy consumption of DNN training. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)* (2023), USENIX Association, pp. 119–139.

Appendices

Appendix A

Reproducibility and Evidence Guide

The writing package contains chapter sources, a checked bibliography, aggregate figures/tables and a record of the prior-draft review. Original chapter drafts are preserved in `previous_drafts/` for comparison. They are historical versions and are not included in the compiled dissertation.

Figure provenance is recorded in the supporting register with the source CSV, generating script, evidence stage and content hash. Existing data files may retain older variable names; captions state the interpretation used in this dissertation. The current campaign is identified as `multiday_20260908`. Its full workload manifests, detailed simulation logs and raw source archive remain in the local research project rather than the shared writing package.

Artefact	Purpose
Workload profile	Source timing, resources, runtime, flexibility and carry-in membership for each simulated job.
Policy decision audit	Candidate information, budgets, selected release and reasons for retaining normal admission.
Replay validation	Complete submissions, starts and completions; timeout and carry-in checks.
Interval carbon output	Common-horizon occupancy, energy and generation-intensity integration under each model.
Daily campaign summary	Matched-date metrics, repetition envelopes and the planned-date completion denominator.
Figure and table register	Mapping from a dissertation visual to its source and empirical status.

The final main-study snapshot is the completed `multiday_20260908` campaign, closed on 10 September 2026. It contains 55 required final replays and 35 policy-date outcomes. Its original manifest is retained alongside the parallel-execution amendment and carry-in correction record. A separate `policy_v2_20260910` directory contains the bounded extension; it is closed and does not overwrite main-study configurations. The dated source manifests

accompanying this manuscript contain full SHA-256 hashes rather than relying on the date or policy label to identify a version.

The reproducibility package distinguishes final results from retained attempts. The first extra Adaptive attempt on 10 December contains 161 timeouts and is preserved in its previous-attempt directory. Legacy 8 December outputs are also preserved but are not accepted as matched initial-state evidence. These records allow corrections to be checked without silently changing the original files.

Only aggregate evidence and the two small engineering-check summaries are included with the writing sources. Raw accounting records, real user identifiers, private configuration and the EVRP reference PDF are not uploaded as part of this dissertation package. Earlier chapter drafts remain historical reference material and are not compiled into the current manuscript.

Appendix B

Complete five-date results

The following tables contain all seven policies on all five dates. They preserve the same evaluation cohort within each date and retain negative and zero-action results. Primary carbon uses median values from three baseline replays and two Adaptive replays; the other policies have one replay per date. Paired waiting is calculated for matching evaluation-job IDs, then summarised across replay pairs. A dagger identifies audited zero-action cases with executable inputs identical to baseline: their differences are replay variation rather than an intentional-delay effect.

Carbon is modelled shared-queue carbon, not per-job meter data. Normalising a shared saving by the evaluation-job count does not uniquely attribute carbon to those jobs. Percentages for alternative occupancy representations are sensitivity checks, not independent measurements. The dates can overlap through carry-in and execution tails; the tables do not form a continuous five-day or annual emissions account.

Table B.1 provides absolute and percentage carbon changes, including the primary whole-cage percentage. The service table separates direct hold, actual added total waiting and concentration of user burden. A P95 of job-paired wait differences is not the difference between two population P95 values. Companion CSVs retain full precision and the extra model/scope fields; the source register is `supporting/final_results_source_manifest_20260910.csv`.

Table B.1: All 35 daily carbon results (2025). $K_C = C_{b,C} - C_{p,C}$ is primary cluster saving (kg CO₂); $g_C = 1000K_C/N$ (g CO₂ per evaluation job). R_C, R_R, R_A are load-related saving percentages for capacity-weighted, node-request upper and distinct allocated-node proxies; $R_{C,w}$ is whole-cluster primary saving percentage. Negative means increase. † marks zero-action replay variability. Policy abbreviations follow the results chapter.

Date	Policy	N (jobs)	K_C (kg)	g_C (g/job)	R_C (%)	$R_{C,w}$ (%)	R_R (%)	R_A (%)
11/13	B1	5806	+1.0936	+0.1884	+1.3835	+0.0603	-0.7234	+1.5710
11/13	B2	5806	+0.4482	+0.0772	+0.5670	+0.0247	-0.2529	-0.4387
11/13	AD	5806	+0.5773	+0.0994	+0.7304	+0.0318	-0.3090	+2.6291
11/13	HY	5806	+0.6269	+0.1080	+0.7931	+0.0345	-0.2946	-0.9804
11/13	HI	5806	+0.2177	+0.0375	+0.2754	+0.0120	-0.2737	-1.7535
11/13	OR†	5806	-0.0019	-0.0003	-0.0024	-0.0001	+0.0016	-0.9431
11/13	LI	5806	+0.0144	+0.0025	+0.0182	+0.0008	-0.0150	-0.3784
11/15	B1	1629	+0.0654	+0.0401	+0.0930	+0.0029	-0.0412	-1.9977
11/15	B2†	1629	+0.0140	+0.0086	+0.0199	+0.0006	-0.0059	-1.0220
11/15	AD	1629	+0.0014	+0.0008	+0.0019	+0.0001	-0.0092	-0.1168
11/15	HY	1629	+0.0613	+0.0377	+0.0873	+0.0027	-0.0378	-1.2631
11/15	HI†	1629	+0.0072	+0.0044	+0.0103	+0.0003	-0.0121	-1.0736
11/15	OR†	1629	+0.0008	+0.0005	+0.0011	+0.0000	+0.0019	-1.2413
11/15	LI†	1629	-0.0004	-0.0003	-0.0006	-0.0000	+0.0007	-0.4629
11/18	B1	5646	+0.6000	+0.1063	+0.5534	+0.0198	-0.0827	+0.7304
11/18	B2	5646	-0.0278	-0.0049	-0.0256	-0.0009	-0.0110	-0.6907
11/18	AD	5646	+0.1838	+0.0326	+0.1696	+0.0061	+0.0011	-1.2976
11/18	HY	5646	+0.1906	+0.0338	+0.1758	+0.0063	+0.0190	-1.3920
11/18	HI	5646	+0.2622	+0.0464	+0.2418	+0.0087	+0.0038	-0.9558
11/18	OR†	5646	-0.0206	-0.0036	-0.0190	-0.0007	-0.0057	-0.6663
11/18	LI	5646	+0.0169	+0.0030	+0.0156	+0.0006	+0.0107	-1.9201

Continued on next page

Table B.1 continued

Date	Policy	N (jobs)	K_C (kg)	g_C (g/job)	R_C (%)	$R_{C,w}$ (%)	R_R (%)	R_A (%)
12/09	B1	3113	+0.2190	+0.0704	+0.2205	+0.0107	-0.6278	-1.0329
12/09	B2 [†]	3113	+0.1012	+0.0325	+0.1019	+0.0049	+0.1894	+1.1534
12/09	AD	3113	+0.3880	+0.1246	+0.3905	+0.0189	-0.0115	-0.9611
12/09	HY	3113	+0.1812	+0.0582	+0.1824	+0.0088	-0.0156	+0.2473
12/09	HI	3113	+0.2840	+0.0912	+0.2859	+0.0138	+0.0574	-1.7321
12/09	OR [†]	3113	-0.1130	-0.0363	-0.1137	-0.0055	+0.0110	-0.4866
12/09	LI [†]	3113	+0.0328	+0.0105	+0.0330	+0.0016	+0.0786	+1.2482
12/10	B1	3210	-3.3071	-1.0302	-2.6427	-0.1284	-4.9396	-1.0636
12/10	B2 [†]	3210	-1.6879	-0.5258	-1.3488	-0.0655	-0.3644	-0.8082
12/10	AD	3210	-1.2113	-0.3774	-0.9680	-0.0470	-0.3399	-0.6495
12/10	HY	3210	-0.8236	-0.2566	-0.6582	-0.0320	-0.4685	+4.4877
12/10	HI	3210	-0.3789	-0.1180	-0.3028	-0.0147	-0.2004	+1.1168
12/10	OR [†]	3210	-1.0098	-0.3146	-0.8069	-0.0392	-0.1389	+1.5631
12/10	LI [†]	3210	-1.7174	-0.5350	-1.3723	-0.0667	-0.3104	-0.9921

Values use four decimal places; signed zero retains the unrounded direction. The companion CSV preserves source precision for all six model–scope combinations, including whole-cluster R and A values and model-specific repeat ranges. Frozen CSV sources and SHA-256 hashes for all new result assets are recorded in `supporting/final_results_source_manifest_20260910.csv`.

Table B.2: All 35 daily wait and burden results (2025). n_D counts directly held jobs; $\bar{D}$ is mean hold. $\bar{\Delta W}$ and $P95(\Delta W)$ summarize paired added total wait; ΔW_0 is the mean among jobs not directly held. U_D, U_+ are the largest user's shares of total hold and positive added wait. Times are minutes per job in the stated cohort. $\dagger$ marks zero-action replay variability.

Date	Policy	n_D (jobs)	$\bar{D}$ (min)	$\bar{\Delta W}$ (min)	$P95(\Delta W)$ (min)	ΔW_0 (min)	U_D (%)	U_+ (%)
11/13	B1	1026	36.35	+35.26	+224.07	-1.38	50.6	50.3
11/13	B2	705	22.02	+21.00	+239.37	-1.22	57.7	57.2
11/13	AD	470	12.51	+11.62	+99.88	-1.06	77.1	75.8
11/13	HY	860	31.70	+30.63	+239.33	-1.26	62.1	61.7
11/13	HI	10	0.32	+0.40	+1.09	+0.05	63.7	41.9
11/13	OR [†]	0	0.00	-0.08	+0.21	-0.08	–	62.8
11/13	LI	2	0.02	+0.04	+0.88	+0.02	50.0	65.3
11/15	B1	324	29.12	+26.45	+213.65	-6.04	52.1	47.6
11/15	B2 [†]	0	0.00	-0.60	+1.21	-0.60	–	31.3
11/15	AD	73	2.00	+3.10	+43.76	-2.42	73.4	61.3
11/15	HY	252	21.80	+19.36	+187.96	-5.59	43.1	39.1
11/15	HI [†]	0	0.00	+0.72	+9.14	+0.72	–	51.5
11/15	OR [†]	0	0.00	+0.00	+3.71	+0.00	–	46.6
11/15	LI [†]	0	0.00	+0.56	+5.79	+0.56	–	48.5
11/18	B1	1423	40.31	+39.57	+221.02	-0.23	61.1	62.1
11/18	B2	260	1.39	+1.07	+3.77	-0.20	51.5	52.7
11/18	AD	64	1.58	+1.11	+1.87	-0.17	37.5	35.3
11/18	HY	586	13.62	+12.79	+194.29	-0.31	67.1	69.3
11/18	HI	5	0.16	+0.15	+3.54	+0.05	50.1	41.8
11/18	OR [†]	0	0.00	+0.64	+5.63	+0.64	–	50.6
11/18	LI	1	0.01	+0.28	+2.30	+0.27	100.0	52.0

Continued on next page

Table B.2 continued

Date	Policy	n_D (jobs)	$\bar{D}$ (min)	$\overline{\Delta W}$ (min)	$P95(\Delta W)$ (min)	ΔW_0 (min)	U_D (%)	U_+ (%)
12/09	B1	820	41.67	+40.92	+363.28	-6.71	26.9	37.9
12/09	B2 [†]	0	0.00	-5.81	+27.63	-5.81	–	26.0
12/09	AD	176	7.54	+28.05	+85.74	+20.68	31.7	29.0
12/09	HY	175	8.73	+23.08	+93.00	+22.70	43.8	39.3
12/09	HI	41	2.00	+13.97	+57.14	+10.33	86.8	32.4
12/09	OR [†]	0	0.00	-7.07	+12.25	-7.07	–	21.7
12/09	LI [†]	0	0.00	+0.66	+44.13	+0.66	–	33.7
12/10	B1	741	30.91	+663.10	+1223.42	+571.36	54.9	48.3
12/10	B2 [†]	0	0.00	+78.60	+154.31	+78.60	–	48.8
12/10	AD	61	1.15	+52.14	+123.13	+51.75	26.7	53.9
12/10	HY	81	4.04	+69.43	+272.14	+65.00	20.8	39.8
12/10	HI	2	0.13	+9.61	+73.42	+9.60	52.3	34.6
12/10	OR [†]	0	0.00	+50.70	+92.20	+50.70	–	54.5
12/10	LI [†]	0	0.00	+85.85	+159.27	+85.85	–	52.7

$\bar{D}$ and $\overline{\Delta W}$ include all evaluation jobs; P95 is a percentile of paired differences. – means undefined hold concentration because total hold is zero. The companion CSV retains source seconds, P99, positive wait, delayed-beyond-runtime counts and maximum-user burden aggregates. No raw user identifiers are included.